Week 6 + 7: Learning from experience

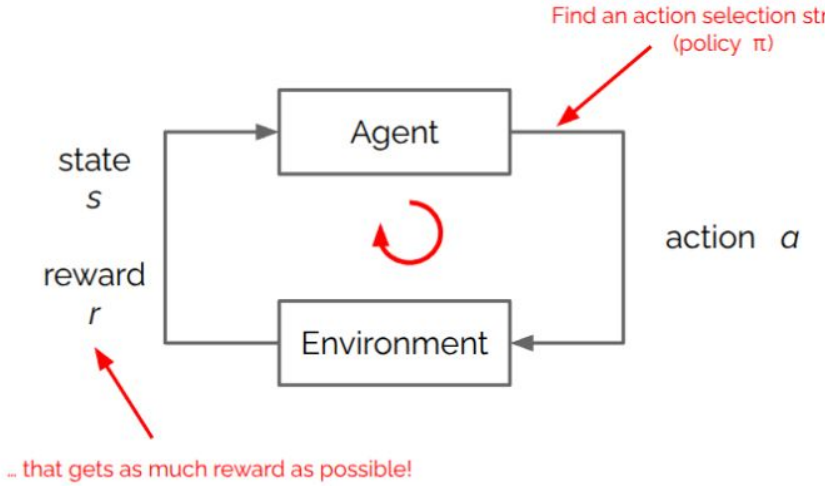
D. M. Pelt | Introduction to Reinforcement Learning

Universiteit
Leiden
The Netherlands

Story so far

- Markov Decision Processes
- Solution method:

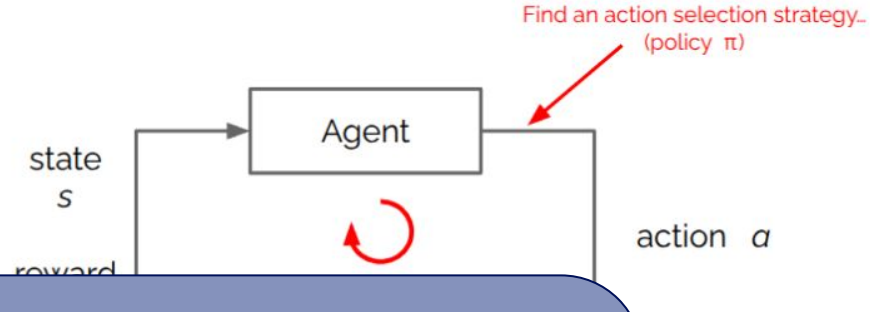
  Dynamic programming



Problem: need complete knowledge about the environment!

Story so far

- Markov Decision Processes
- Solu



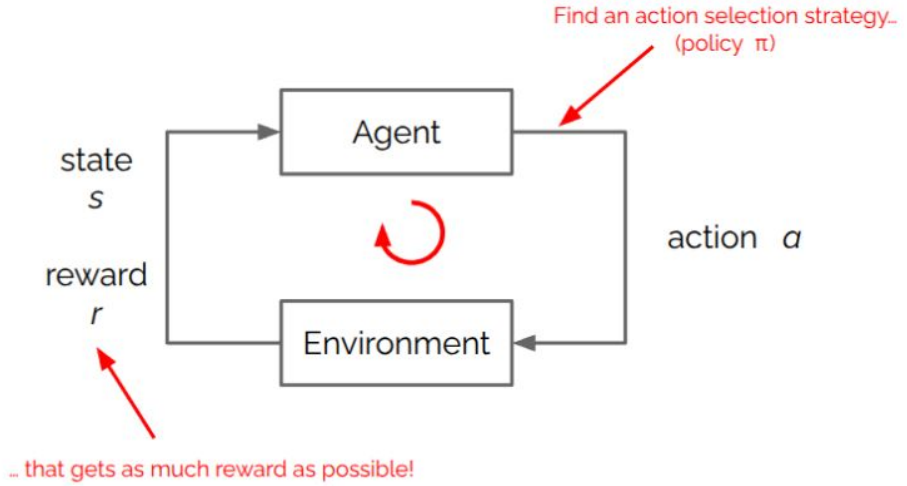
Can we learn from **experience** only?

Problem: need complete knowledge about the environment!

Story so far

MDP:

- State space
- Action space
- ~~Transition function~~
- ~~Reward function~~
- Discount parameter

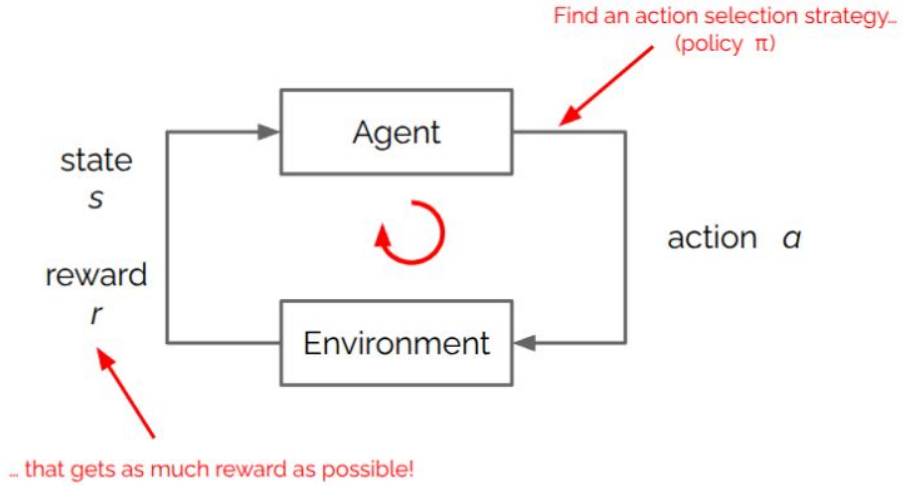


Can we still learn a good policy?

Story so far

MDP:

- State space
- Action space
- ~~Transition function~~
- ~~Reward function~~
- Discount parameter



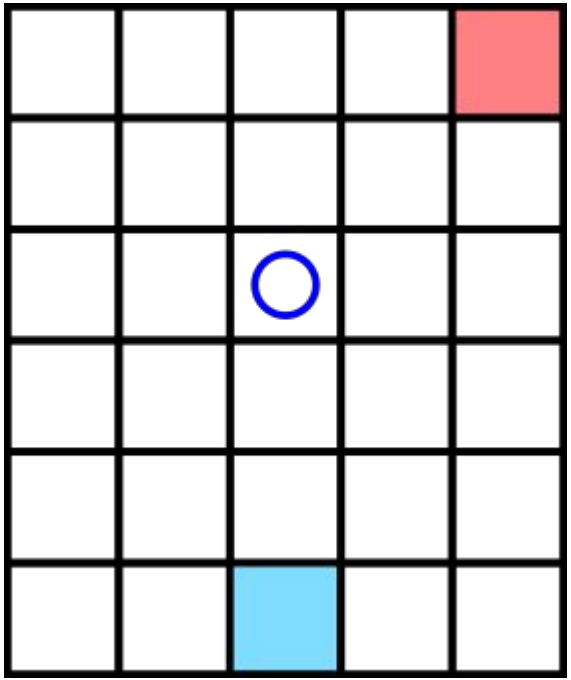
Why not estimate transition function and reward function based on experience?

Week 12 -> Model-based RL

Start simple

- We have a **policy π**
- Task: estimate **value of states:** $v_\pi(s)$

What can we do?

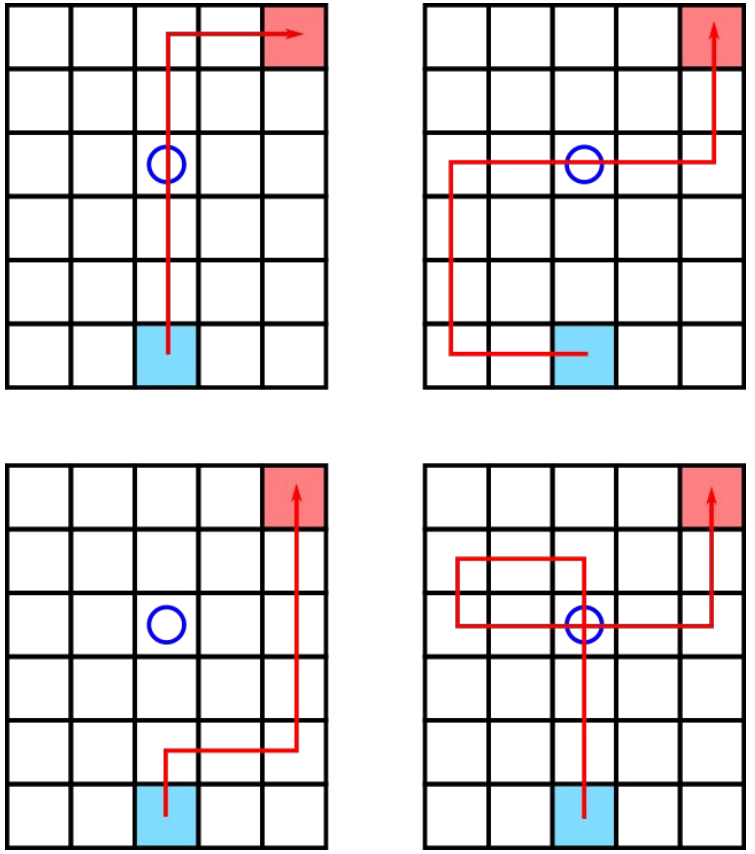


Start simple

- We have a **policy π**
- Task: estimate **value of states:** $v_\pi(s)$

- Just generate episodes!
- Basic idea of MC methods:

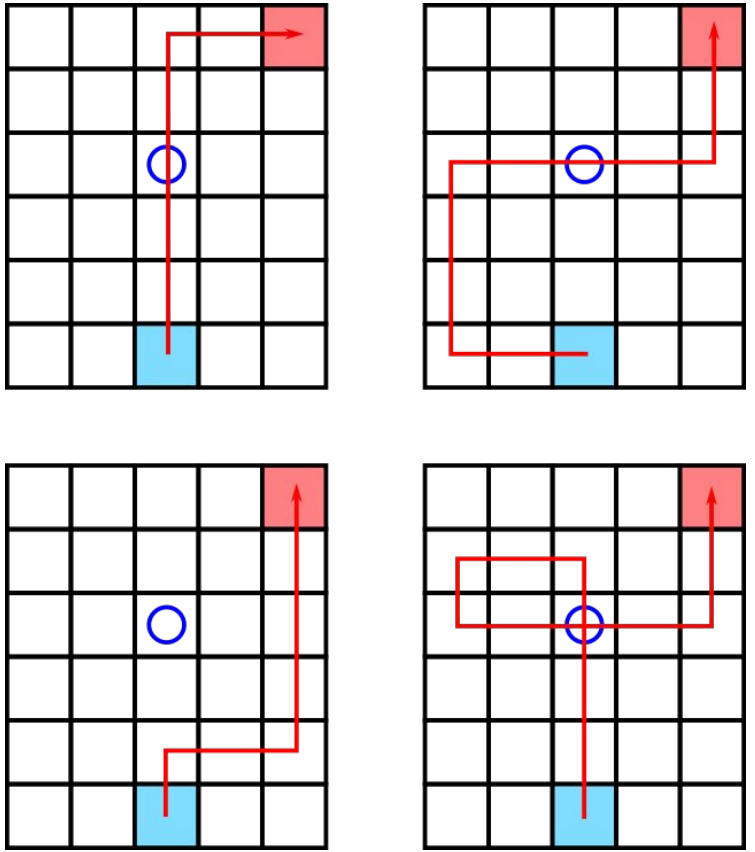
Average observed returns



First-visit vs every-visit

- For each episode, several possibilities:
  - s is **not visited**
  - s is visited **once**
  - s is visited **multiple** times
- Two groups:
  - **First-visit** MC methods
  - **Every-visit** MC methods

Today, only discuss first-visit methods



First-visit MC method to learn state-values

We have a **policy** π, want to estimate **value of states**: $v_\pi(s)$

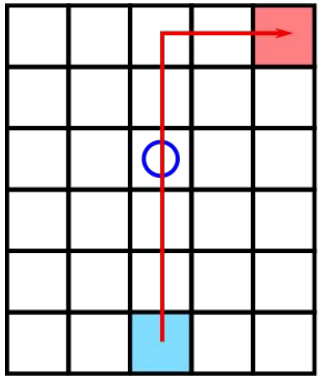
Initialize $v_\pi(s)$ arbitrarily

Loop:

- Generate an episode
- For each state visited in the episode, in reverse order:
  - If this was first time that state was encountered this episode:

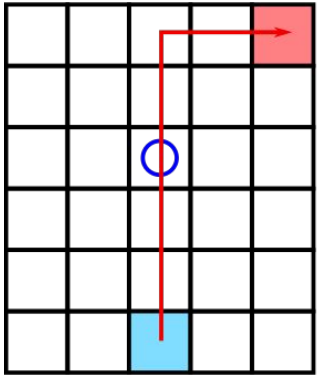
    Update the average $v_\pi(s)$ with the **cumulative future discounted reward** of that state in this episode

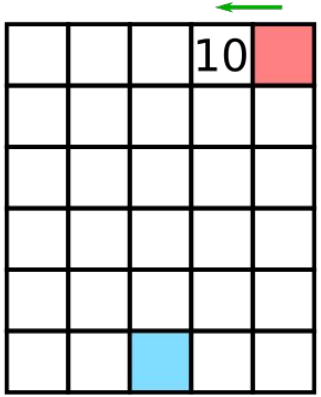
First-visit MC method to learn state-values



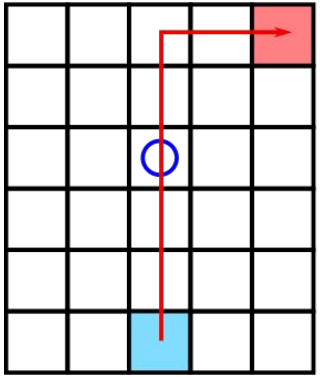
- Reward for reaching goal: 10
- Reward for all other actions: -1
- Discount parameter: 1

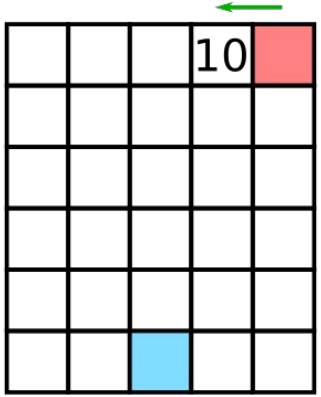
First-visit MC method to learn state-values

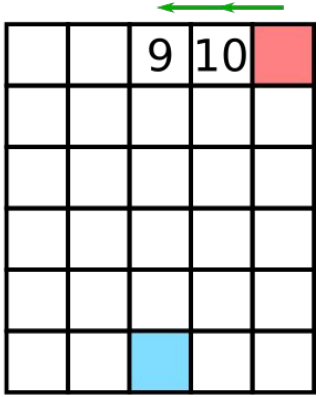




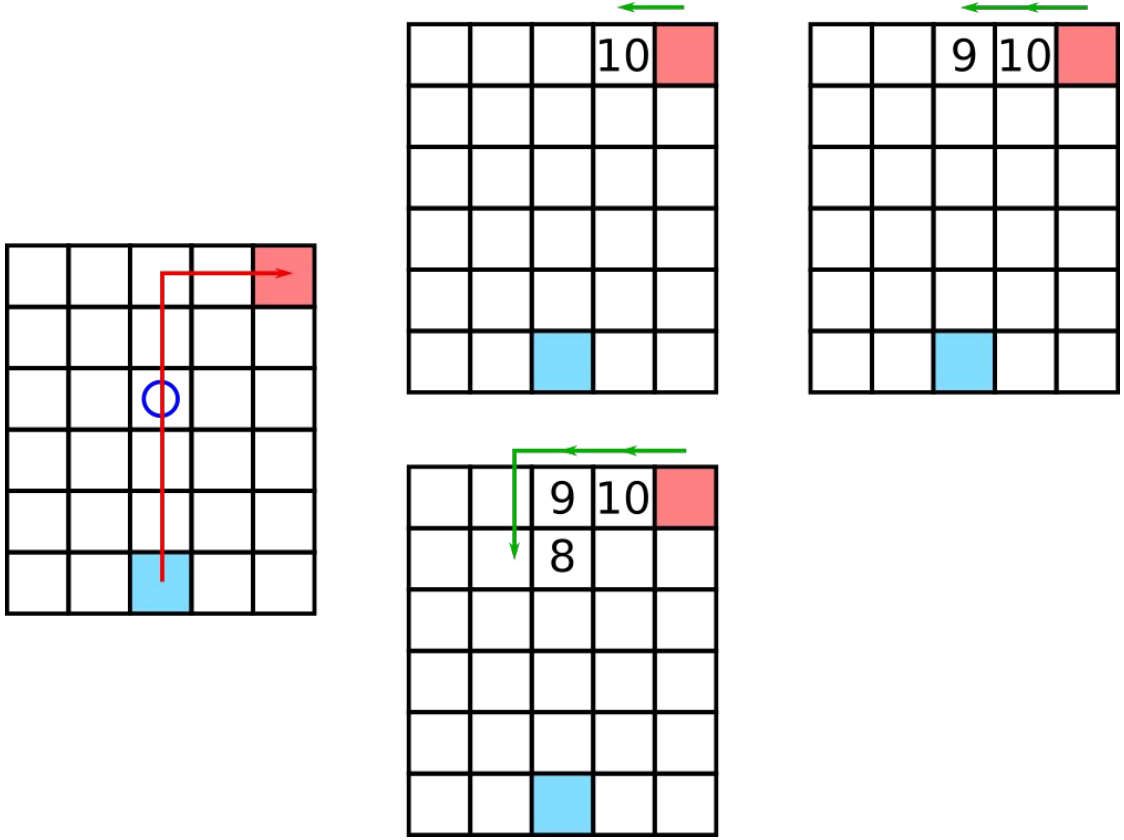
First-visit MC method to learn state-values



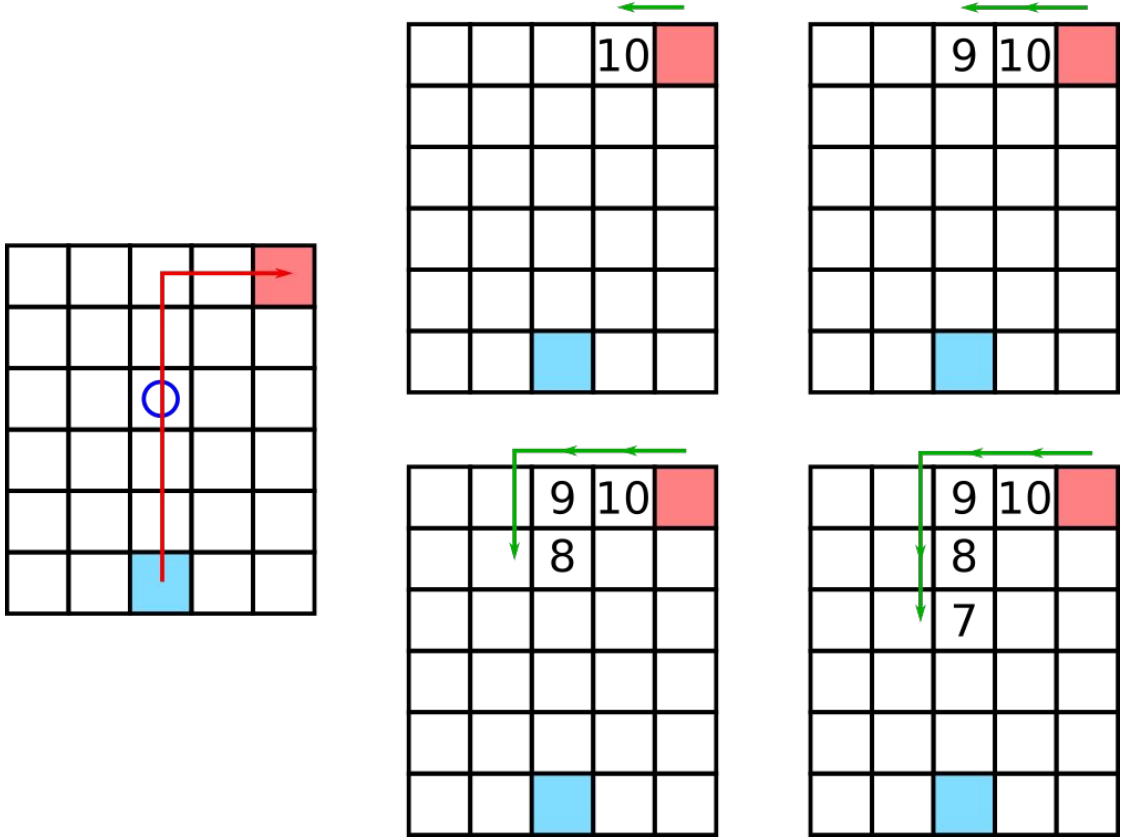




First-visit MC method to learn state-values

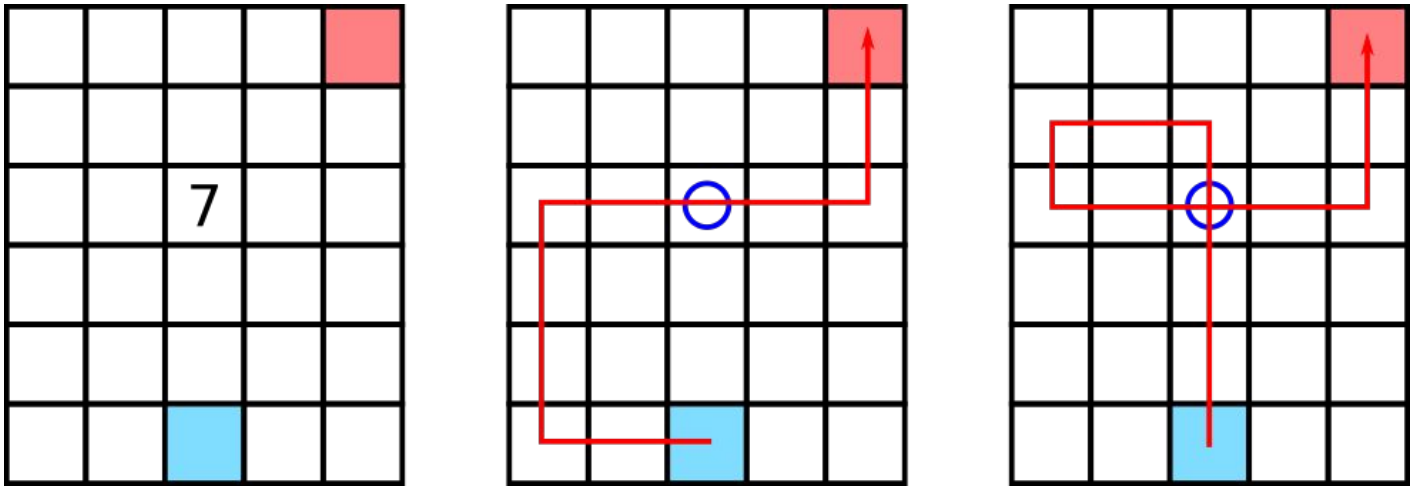


First-visit MC method to learn state-values



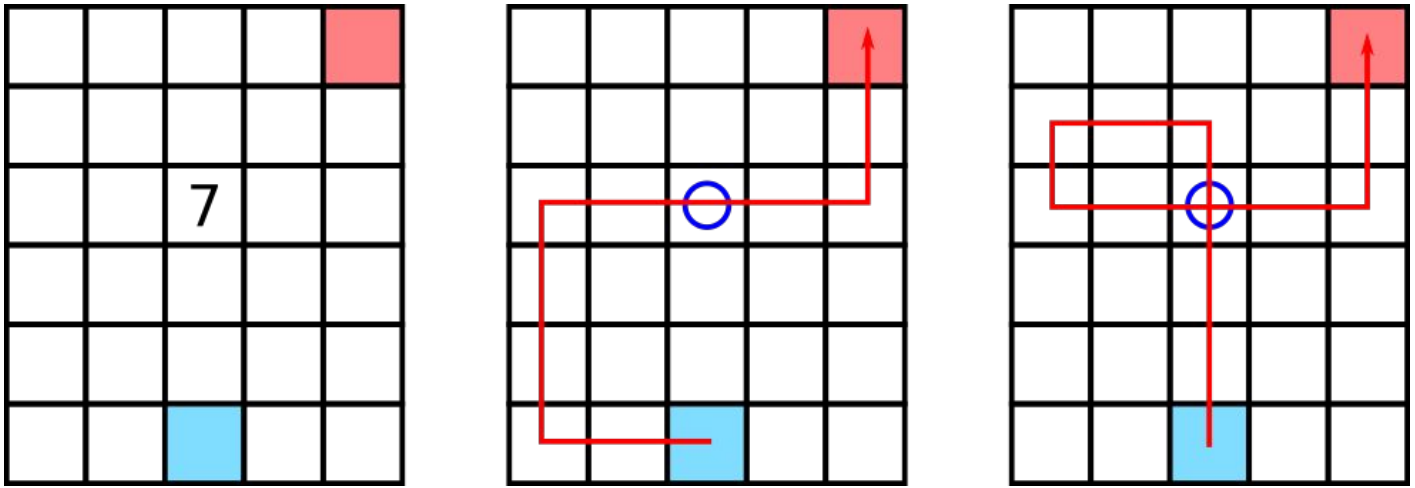
First-visit MC method to learn state-values

We now have $v_\pi(s) \approx 7$. What will it be after each of the next two episodes?



First-visit MC method to learn state-values

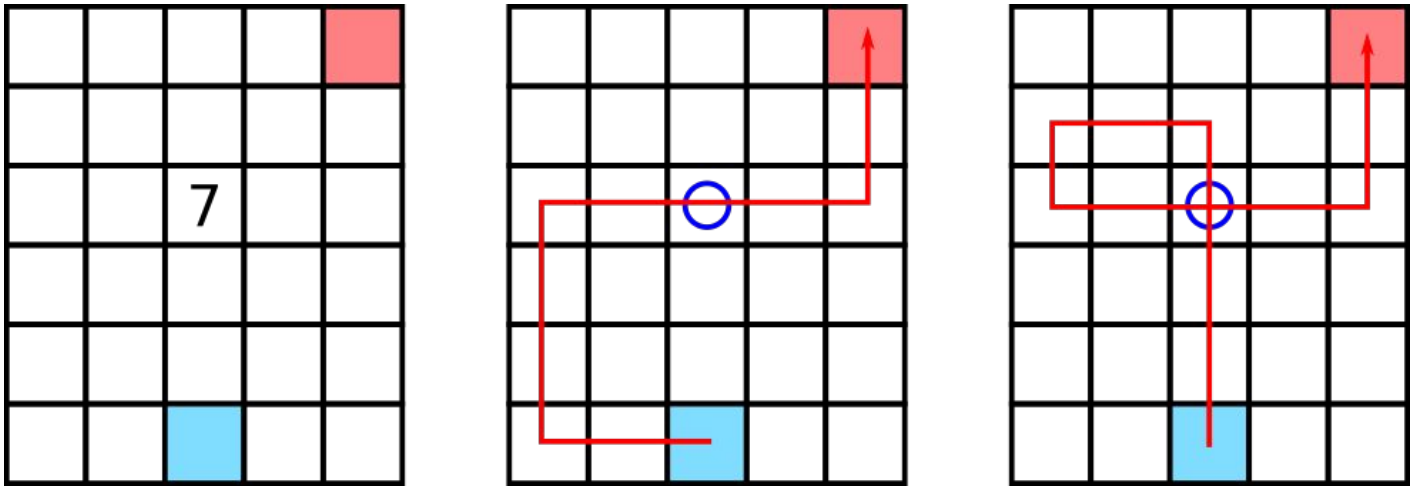
We now have $v_\pi(s) \approx 7$. What will it be after each of the next two episodes?



$$v_\pi(s) \approx (7 + 7)/2 = 7$$

First-visit MC method to learn state-values

We now have $v_\pi(s) \approx 7$. What will it be after each of the next two episodes?



$$v_\pi(s) \approx (7 + 7 + 1)/3 = 5$$

First-visit MC method to learn state-values

We have a **policy** π, want to estimate **value of states**: $v_\pi(s)$

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow$ average$(Returns(S_t))$

Estimating action values

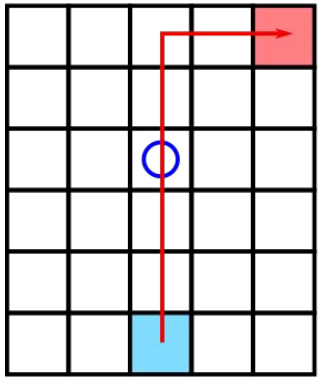
- Usually, state values are **not really what we want**
- We want to learn **which action to take** in a certain state!
- Instead of state values, learn values of **state-action pairs**:

$$q_\pi(s, a)$$

- **MC methods are very similar** to those for action values: just average over state-action pairs

Estimating action values - exploration

- Problem: many state-action pairs can be unvisited!
- MC averages not available if pair is unvisited
- Simple solution, **exploring starts**:
  - Let episode start at random state-action pair
- Problem: often not be possible in practice
- More general solution later



MC with policy iteration

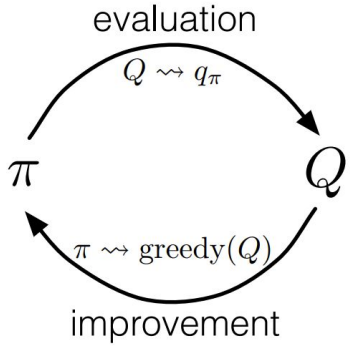
- Often, we want to learn a good policy, i.e. what actions to take
- For MC methods, we can use generalized policy iteration!
- Iteratively:
  - For current policy π, compute action values $q_\pi(s, a)$
  - Choose better policy π given $q_\pi(s, a)$
- Possibility: $\pi(s) = \text{argmax}_a\ q_\pi(s, a)$

MC with policy iteration

- Often, we want to learn a good policy, i.e. what actions to take
- For MC methods, we can use generalized policy iteration!
- Iteratively:
  - For current policy π, compute $q_\pi(s, a)$
  - Choose π given $q_\pi(s, a)$
- Possibility: $\pi(s) = \text{argmax}_a\ q_\pi(s, a)$

Policy evaluation

Policy improvement



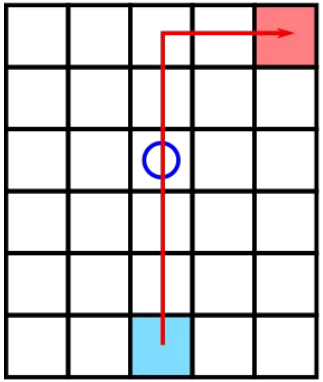
Policy evaluation with a finite number of episodes

- To compute $q_\pi(s, a)$ given π, we need infinitely many episodes
- Can we work with a finite number of episodes?
  - More generally: choose when to do evaluation and improvement
- Two solutions:
  - Approximate $q_\pi(s, a)$ and make sure error is small
  - Just make a few steps toward $q_\pi(s, a)$

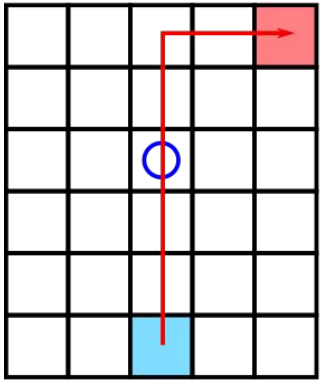
Policy evaluation with a finite number of episodes

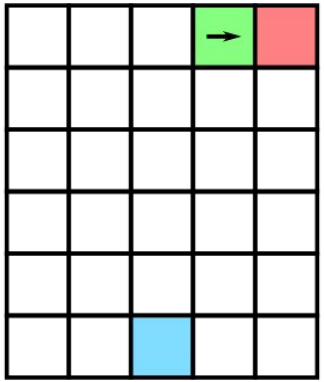
- Just make a few steps toward $q_\pi(s, a)$
- For MC methods, a natural way is **episode-based**
- After an episode:
  - Update $q_\pi(s, a)$ for pairs visited during episode
  - Update π for states visited during episode

Example



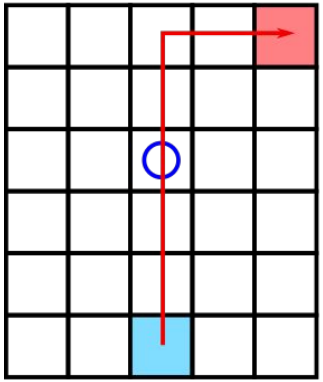
Example

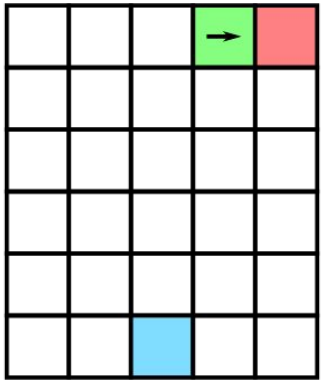


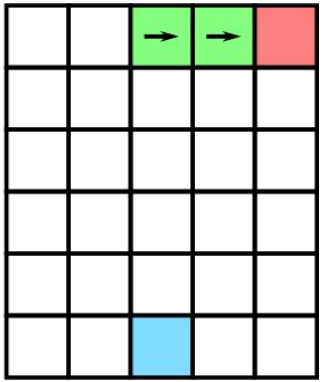


Update action values

Example

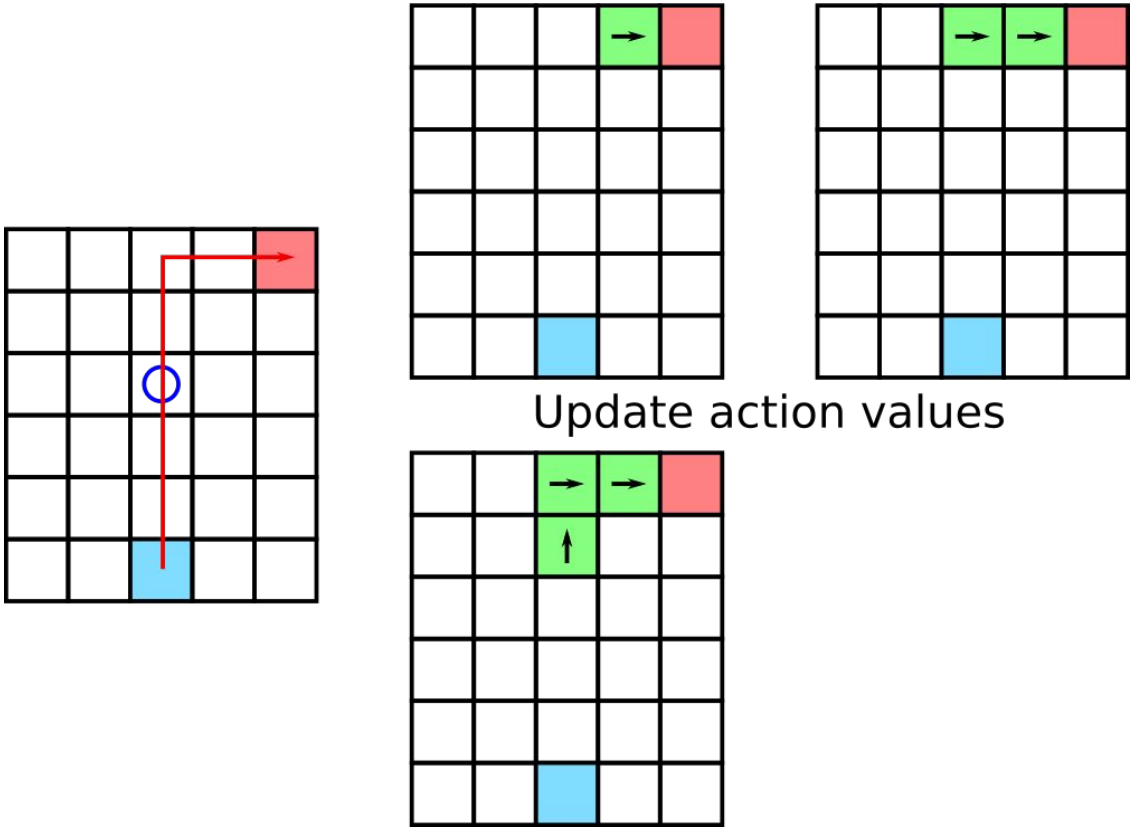




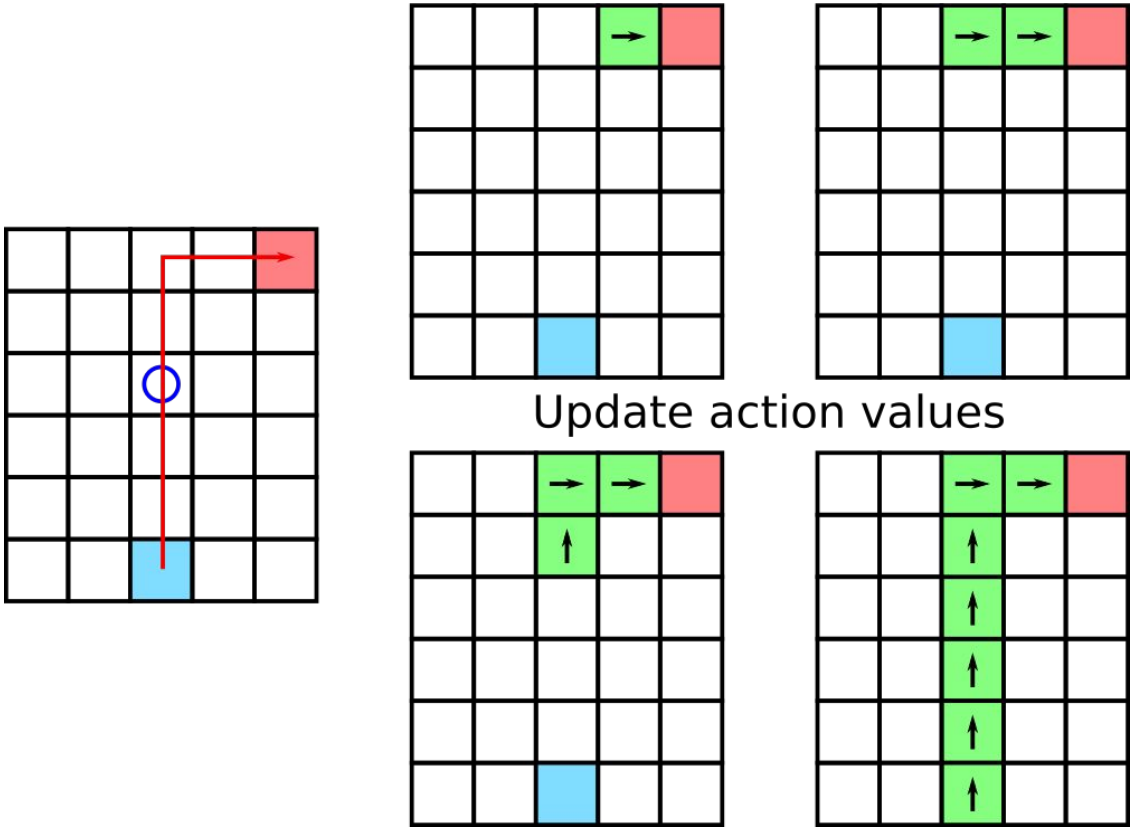


Update action values

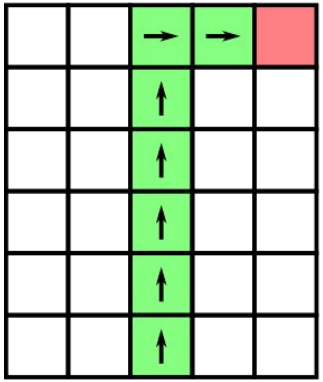
Example



Example

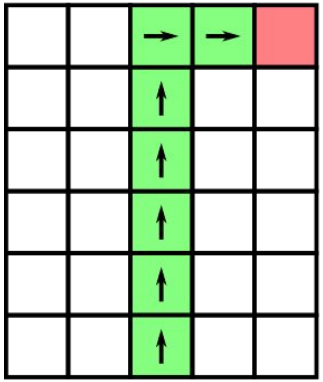


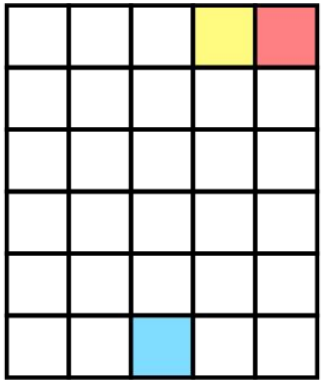
Example



Update policy

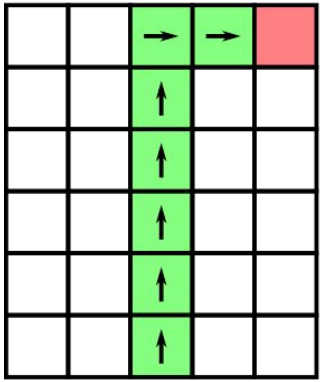
Example

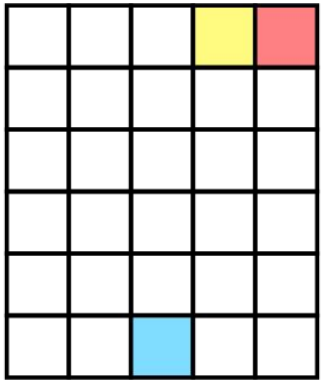


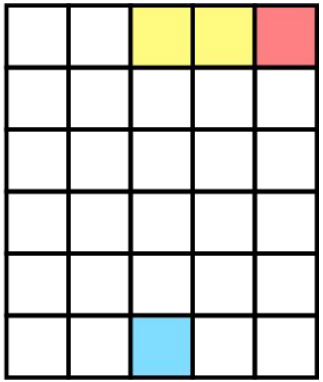


Update policy

Example

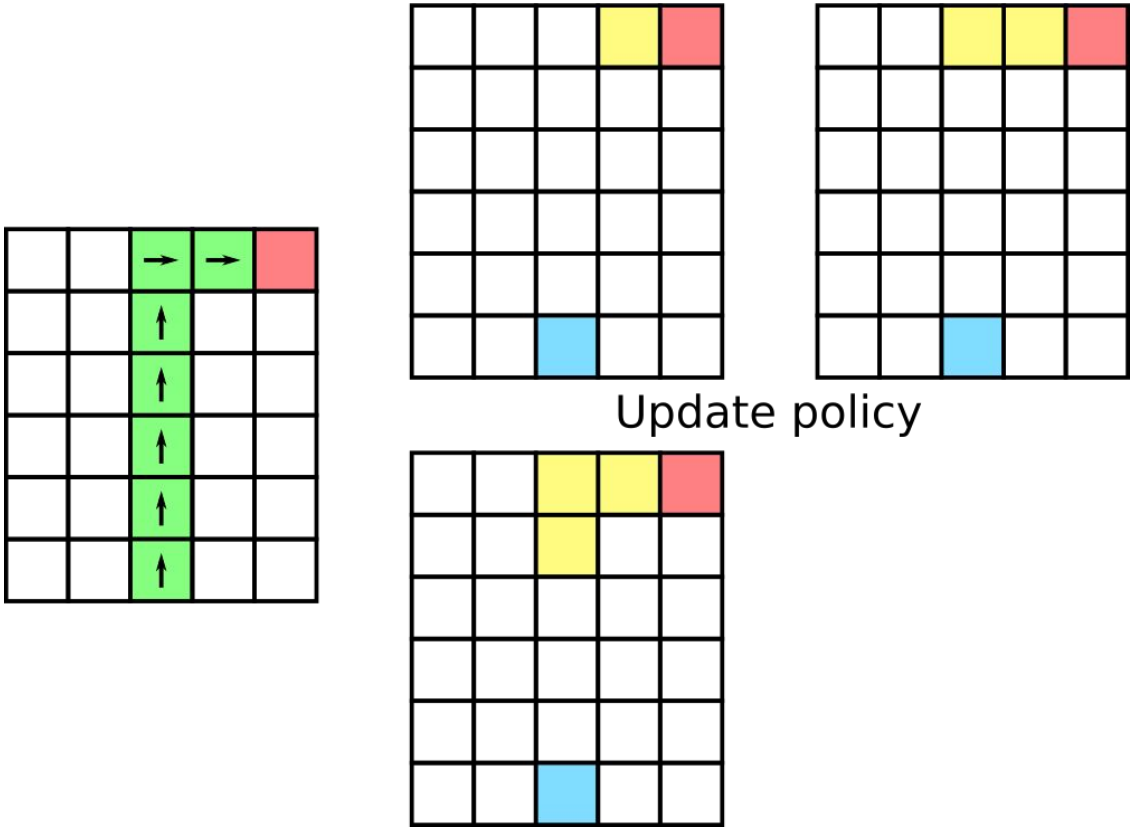




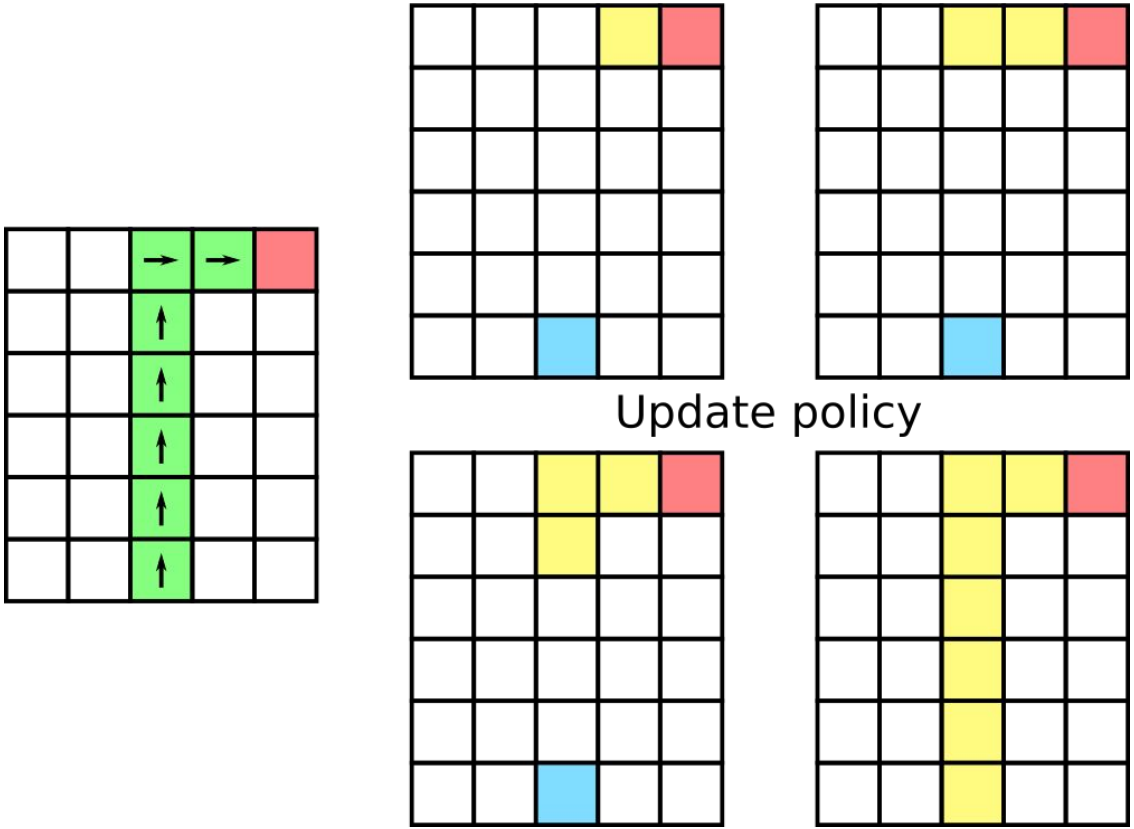


Update policy

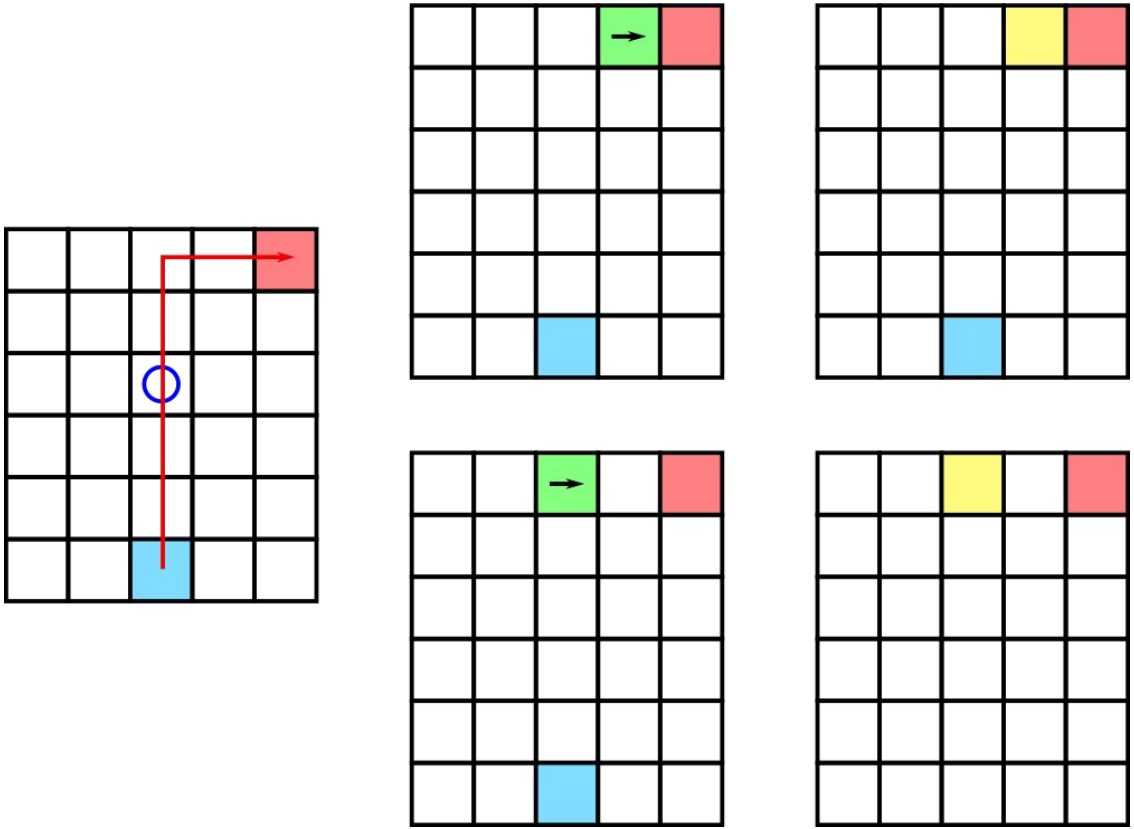
Example



Example



Example



Monte Carlo with Exploring Starts

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

Break

Removing exploring starts

- What if we don't want to (or can't!) use exploring starts?
- Problem: many state-action pairs unvisited
- Recall: exploration vs exploitation
- Solution: make sure all pairs have a non-zero probability
- ε-greedy policy instead of $\pi(s) = \text{argmax}_a\, q_\pi(s, a)$

Removing exploring starts

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε-soft policy

$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

On-policy vs off-policy

- Previous method is an **on-policy** method:

The policy used for generating episodes is also the policy that is optimized

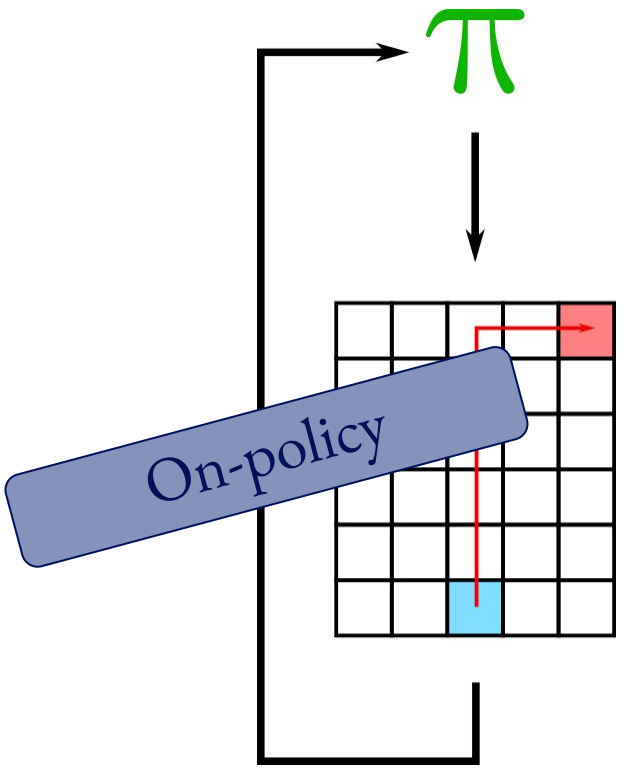
- Another possibility is **off-policy** learning:

Policy being optimized is not used for generating episodes to learn from

On-policy vs off-policy

π

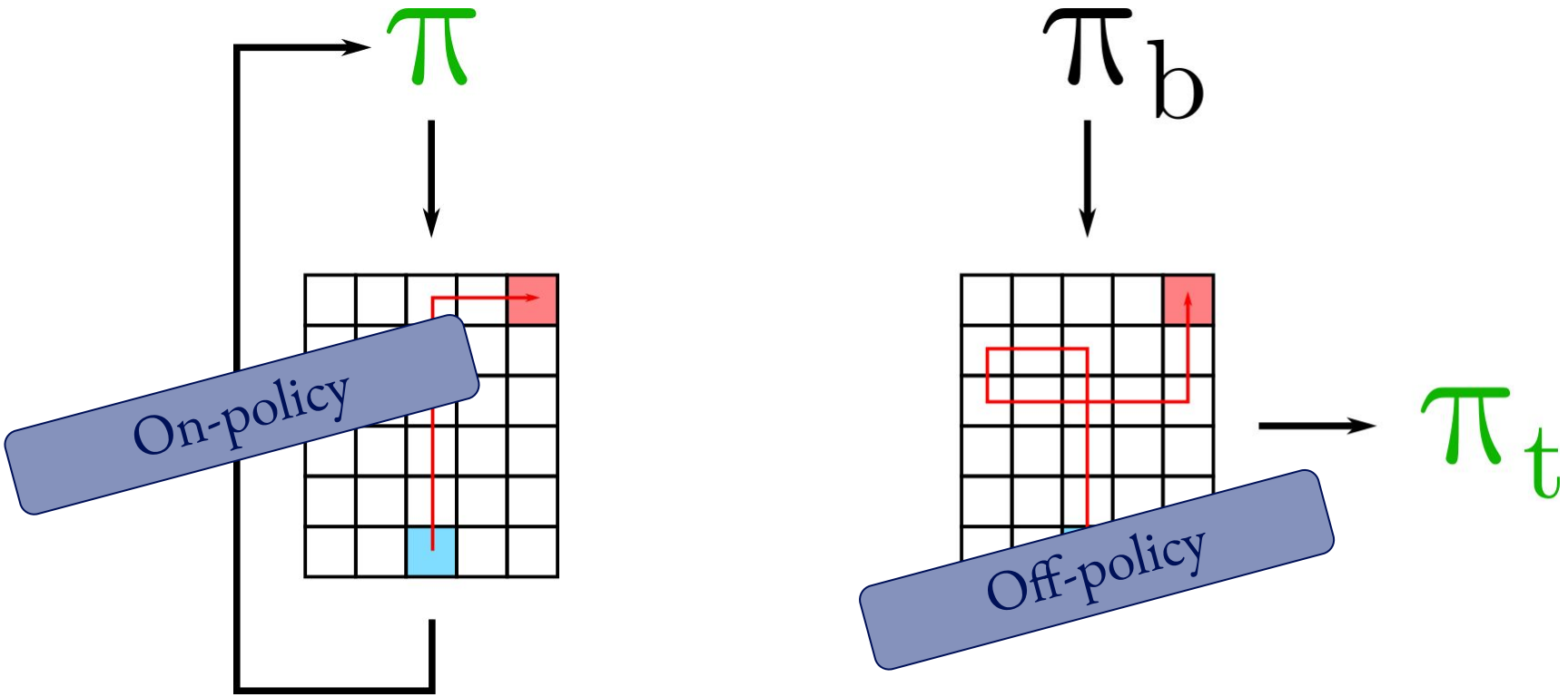
On-policy vs off-policy



π_b

π_t

On-policy vs off-policy



On-policy vs off-policy

- On-policy:
  - Simpler and easy to use
  - Can be suboptimal (have to keep exploring)

- Off-policy:
  - More powerful and general
  - More complicated
  - Can be slower to converge

Start simple again: off-policy prediction

- We have a **behaviour policy b** and **target policy π**
- Task: estimate **value of actions:** $q_\pi(s, a)$
- First assumption: **coverage**
  - Every action in π is taken, at least occasionally, under b
- π can be fully greedy, only b has to keep exploring

Start simple again: off-policy prediction

- We have a **behaviour policy b** and **target policy π**
- Task: estimate **value of actions:** $q_\pi(s, a)$
- Problem: probability of action occurring is different in π and b

Start simple again: off-policy prediction

- We have a **behaviour policy b** and **target policy π**
- Task: estimate **value of actions:** $q_\pi(s, a)$
- Problem: probability of action occurring is different in π and b

Simply averaging returns would result in $q_b(s, a)$!

- Solution: **importance sampling**

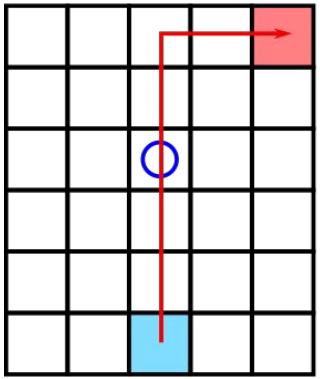
Importance sampling

$$\begin{aligned}
&\Pr\{A_t, S_{t+1}, A_{t+1}, \ldots, S_T \mid S_t, A_{t:T-1} \sim \pi\} \\
&\quad = \pi(A_t|S_t)p(S_{t+1}\,|\,S_t, A_t)\pi(A_{t+1}|S_{t+1}) \cdots p(S_T\,|\,S_{T-1}, A_{T-1}) \\
&\quad = \prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}\,|\,S_k, A_k),
\end{aligned}$$

$$\rho_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}\,|\,S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}\,|\,S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}.$$

Importance sampling

$$\rho_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k,A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k,A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}.$$



$q_\pi(s, a) \approx \text{average}\ (\ \rho\ G\ )$

Importance sampling

$$q_\pi(s, a) \approx \text{average}(\rho\, G)$$

$$\frac{\sum_{\text{episodes}} \rho G}{\text{Number of times (s, a) occurred}}$$

$$\frac{\sum_{\text{episodes}} \rho G}{\sum_{\text{episodes}} \rho}$$

ordinary

weighted

Off-policy MC evaluation

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π
Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
    $Q(s,a) \in \mathbb{R}$ (arbitrarily)
    $C(s,a) \leftarrow 0$

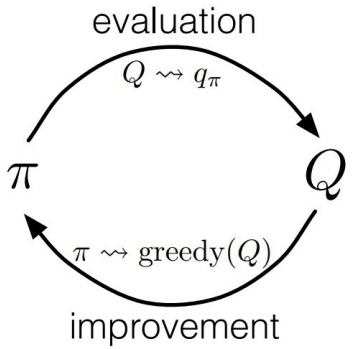
$$\sum_{\text{episodes}} \rho$$

Loop forever (for each episode):
    $b \leftarrow$ any policy with coverage of π
    Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    $W \leftarrow 1$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$, while $W \neq 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
        $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
        $W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

ρ

Off-policy MC evaluation

How do we use this to find good policies?



Generalized policy iteration!

Off-policy MC control

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
$Q(s,a) \in \mathbb{R}$ (arbitrarily)
$C(s,a) \leftarrow 0$
$\pi(s) \leftarrow \arg\max_a Q(s,a)$ (with ties broken consistently)

Loop forever (for each episode):
$b \leftarrow$ any soft policy
Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
$W \leftarrow 1$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)
If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
$W \leftarrow W \frac{1}{b(A_t|S_t)}$

Problems with MC methods

- MC methods **wait for the end of an episode** to learn things
- Can be problem for long episodes, or environments without episodes
- Does not match how humans learn

Can we remove this waiting?

Yes! Using **Temporal-Difference Learning**

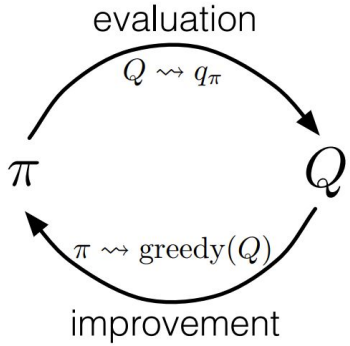
Summary

- Learning from experience, without knowledge about environment
- Monte Carlo methods (MC): run episode, average returns
- Use generalized policy iteration to learn good policies
- On-policy vs off-policy methods
- Problem: Need to wait for end of episode

Can we solve this problem?

Where are we now?

- Goal: find good policies for sequential decision-making problems
- General approach: Generalized policy iteration
  - For given policy, estimate state or action values
  - Given state/action values, improve policy



Dynamic programming

+ Bootstrap

DP

- Need model

Input: a policy π(a|s), an MDP (p(s'|s,a), r(s,a,s'), γ)

Algorithm:

- Initialize v(s)=0 for all s
- Sweep through all states, updating according to Bellman Equation:

$$v^{\pi}(s) = \mathbb{E}_{a \sim \pi(a|s)} \mathbb{E}_{s' \sim p(s'|a,s)} \left[r(s,a,s') + \gamma \cdot v^{\pi}(s') \right]$$

- Until v(s) converges

Monte Carlo

- Have to wait for outcome

MC

+ Learn from experience

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow$ average$(Returns(S_t))$

Temporal difference learning

+ Bootstrap

DP

- Need model

- Have to wait for outcome

MC

+ Learn from experience

+ Bootstrap

TD

+ Learn from experience

Temporal difference prediction

- Start simple again! Have a **policy π,** estimate **value of state:** $v_\pi(s_t)$
- A simple MC method update could be:

$$v_\pi(s_t) \leftarrow (1 - \alpha)\, v_\pi(s_t) + \alpha\, G_t$$

$$v_\pi(s_t) \leftarrow v_\pi(s_t) + \alpha\, [\, G_t - v_\pi(s_t)\,]$$

- Problem: G_t only known after episode ends!

Temporal difference prediction

- Simple MC: $v_\pi(s_t) \leftarrow v_\pi(s_t) + \alpha [G_t - v_\pi(s_t)]$
- Problem: G_t only known after episode ends!

Can we find a solution?

- The role of G_t above is to give an estimate of the $v_\pi(s_t)$ value

Could we use something else as an estimate?

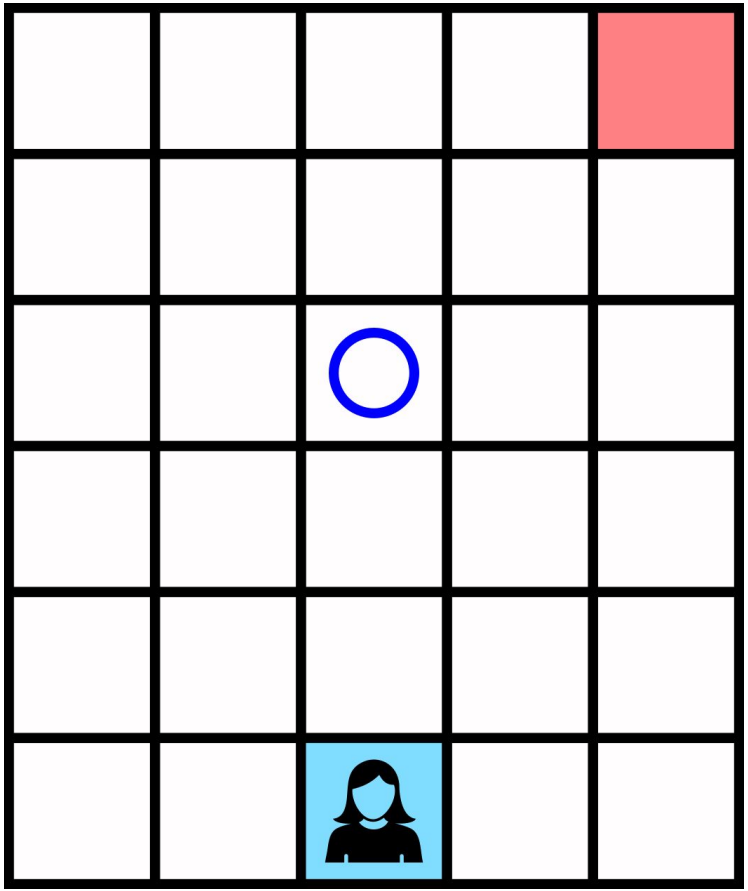
Temporal difference prediction

- Simple MC: $v_\pi(s_t) \leftarrow v_\pi(s_t) + \alpha [G_t - v_\pi(s_t)]$
- Problem: G_t only known after episode ends!
- Solution: instead of G_t, create target based on next step:

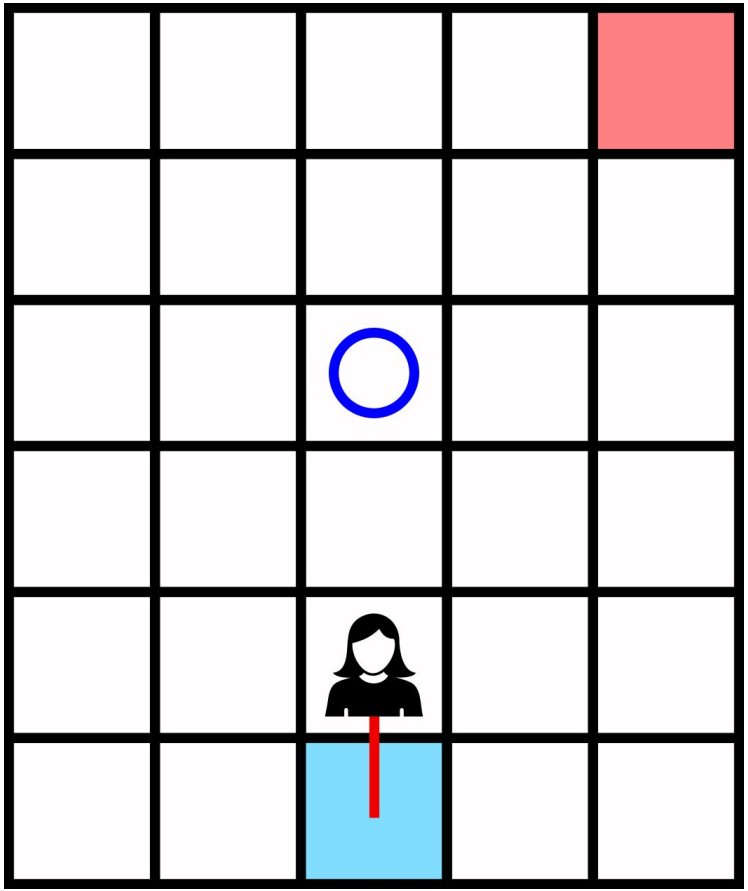
$$v_\pi(s_t) \leftarrow v_\pi(s_t) + \alpha [R_{t+1} + \gamma v_\pi(s_{t+1}) - v_\pi(s_t)]$$

- **TD(0)**, or **one-step TD**

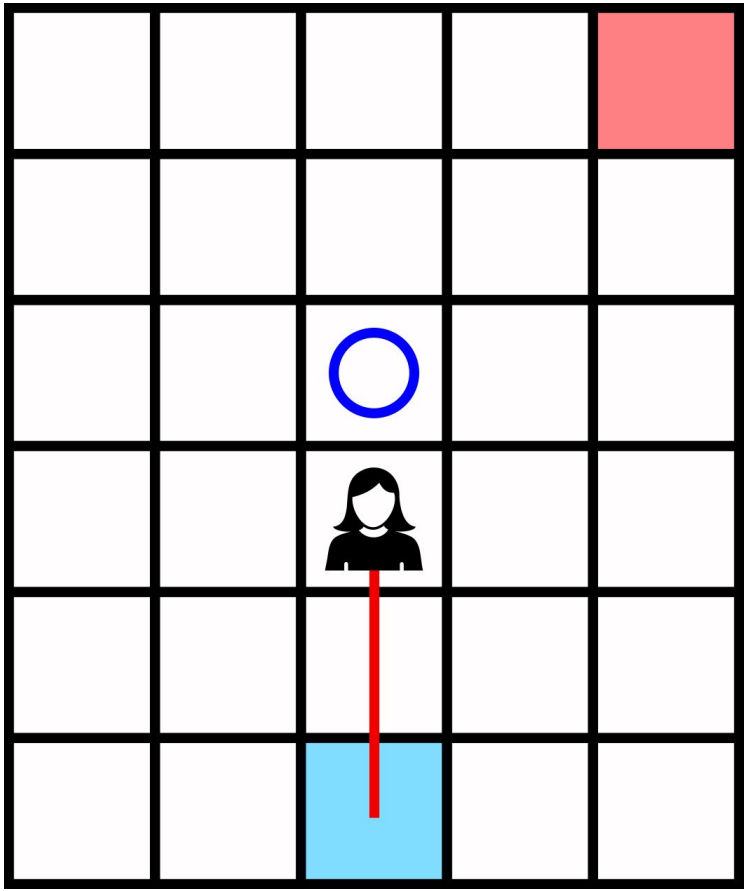
MC evaluation



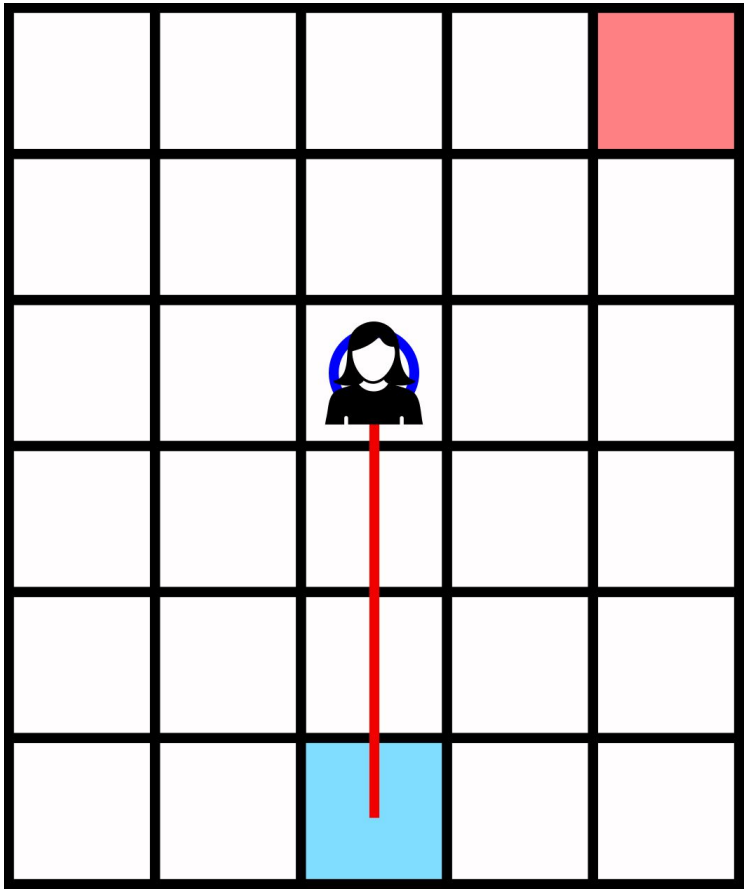
MC evaluation



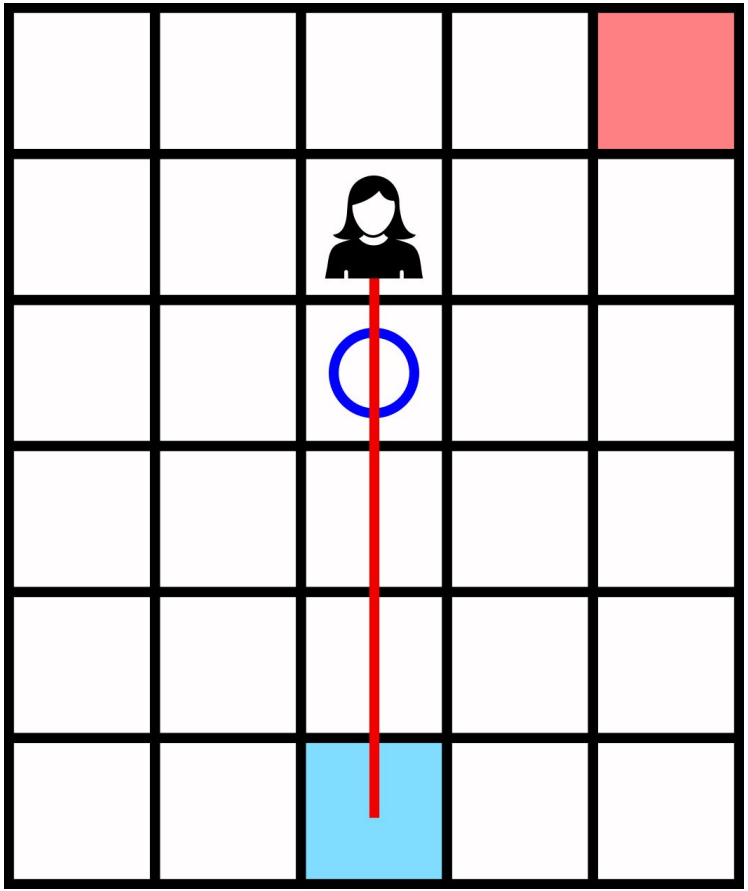
MC evaluation



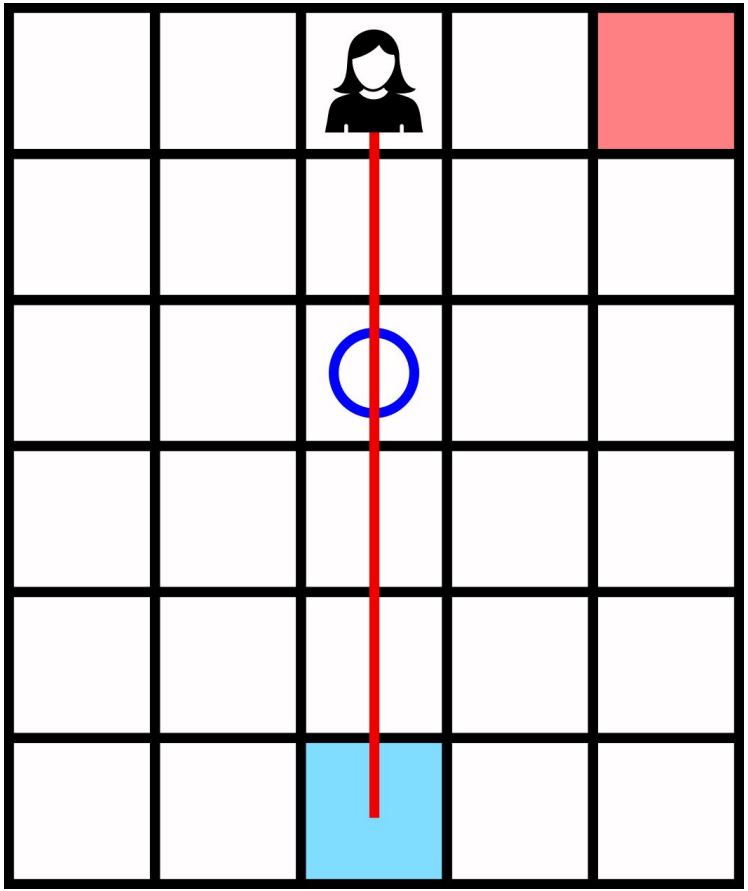
MC evaluation



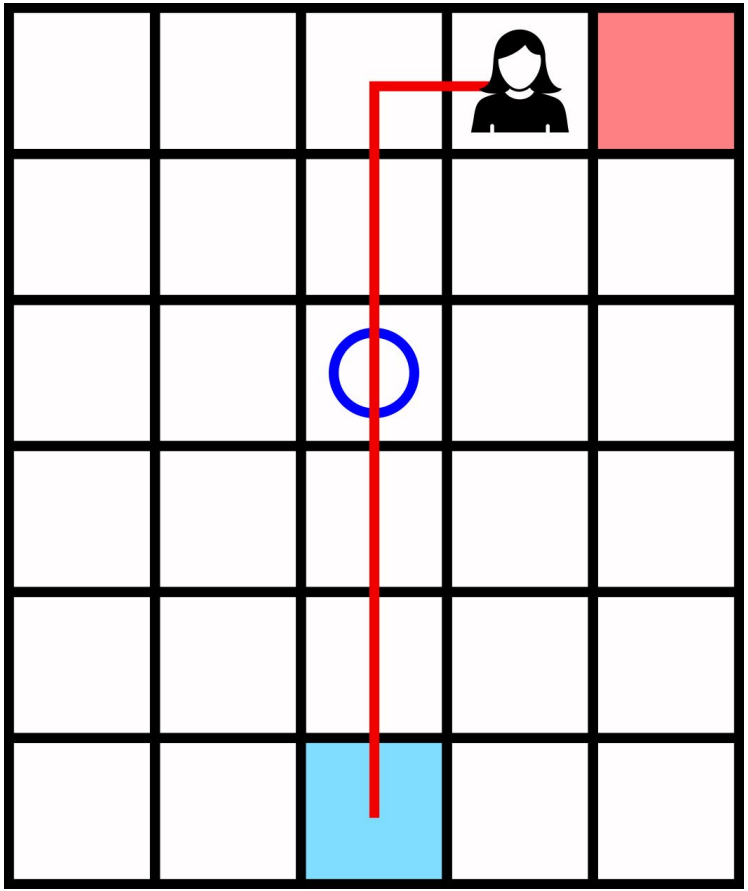
MC evaluation



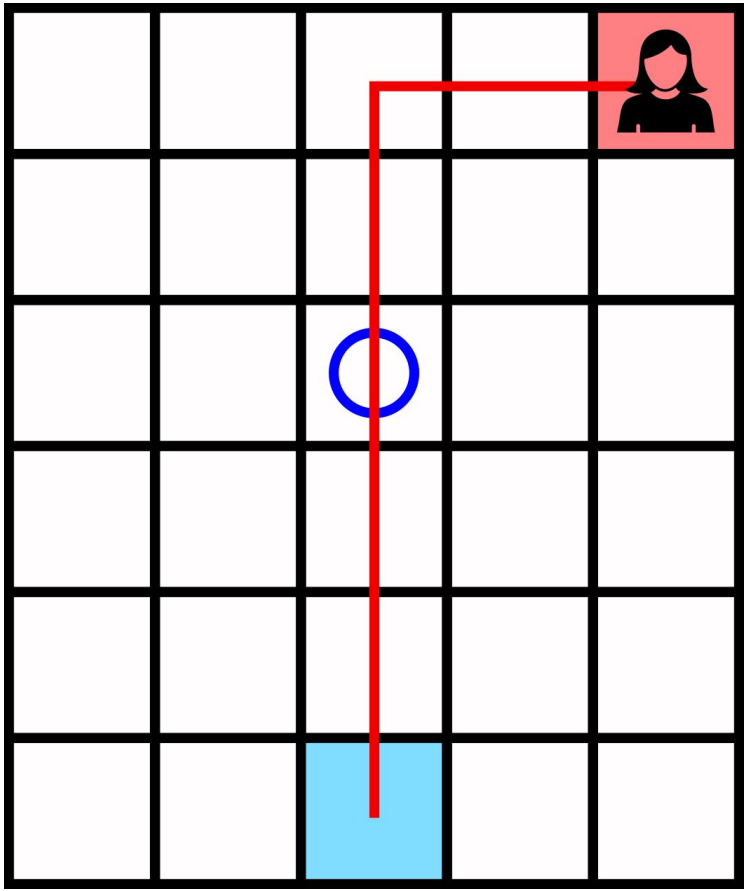
MC evaluation



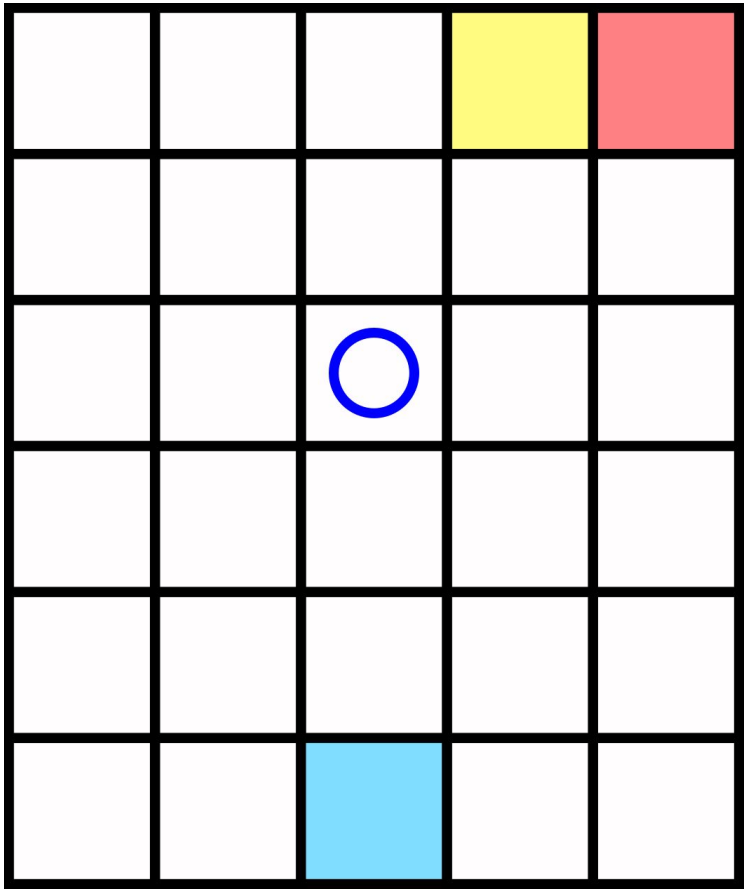
MC evaluation



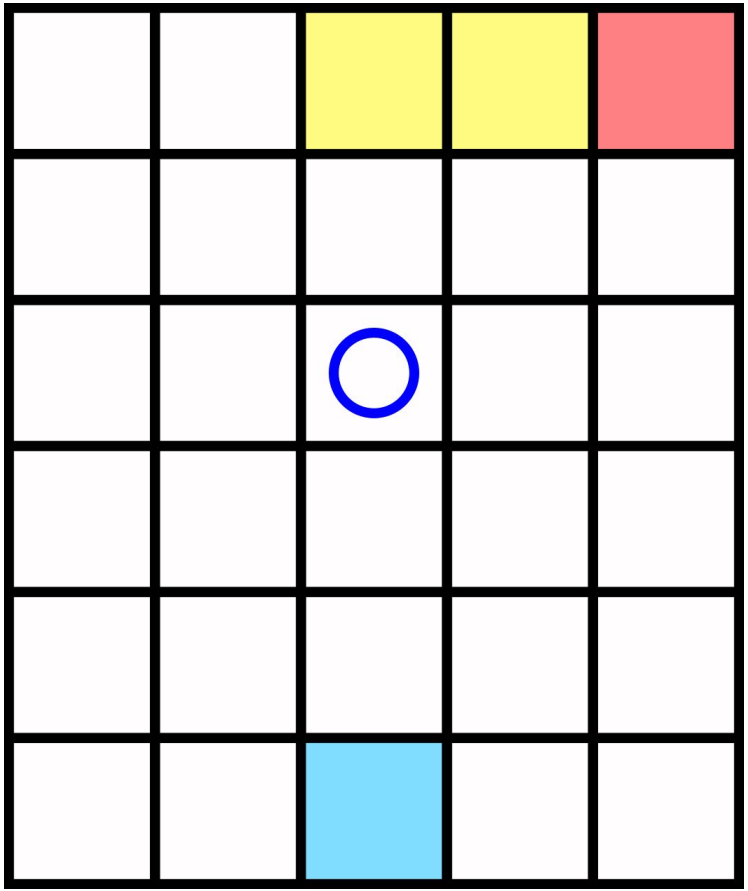
MC evaluation



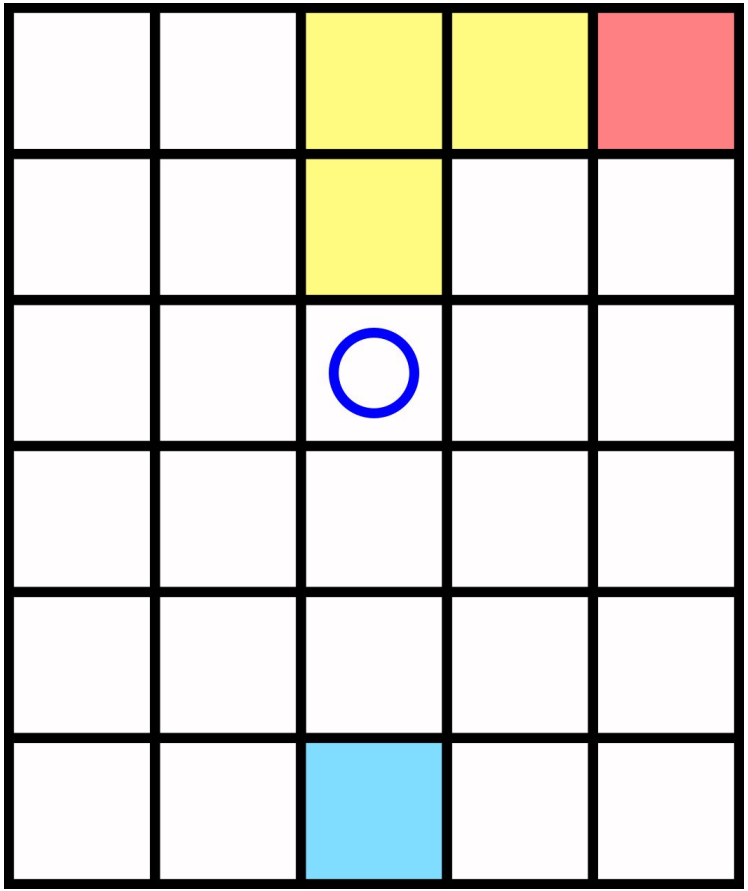
MC evaluation



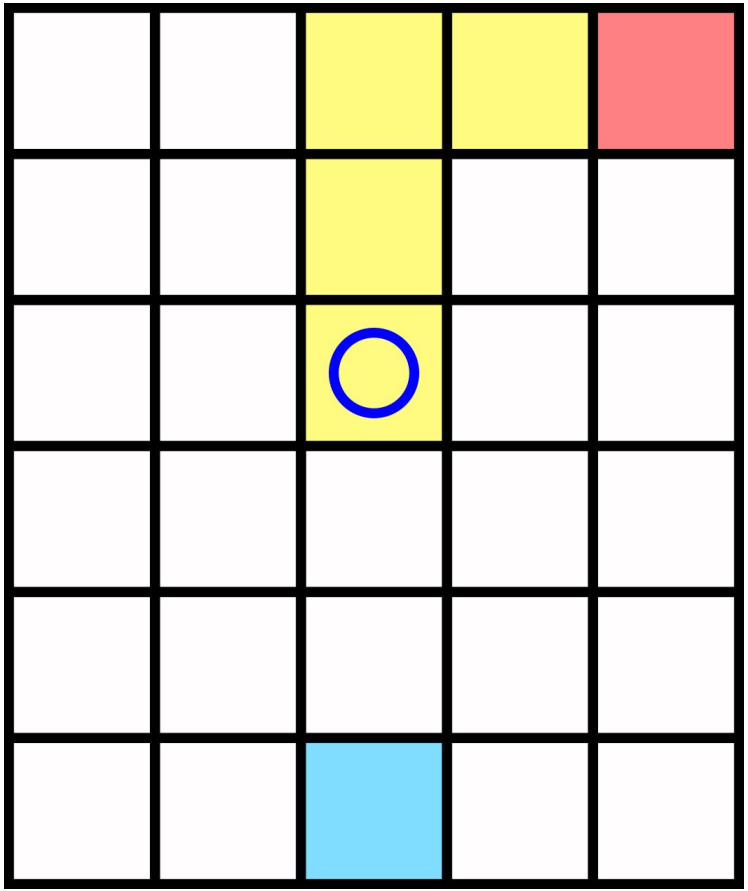
MC evaluation



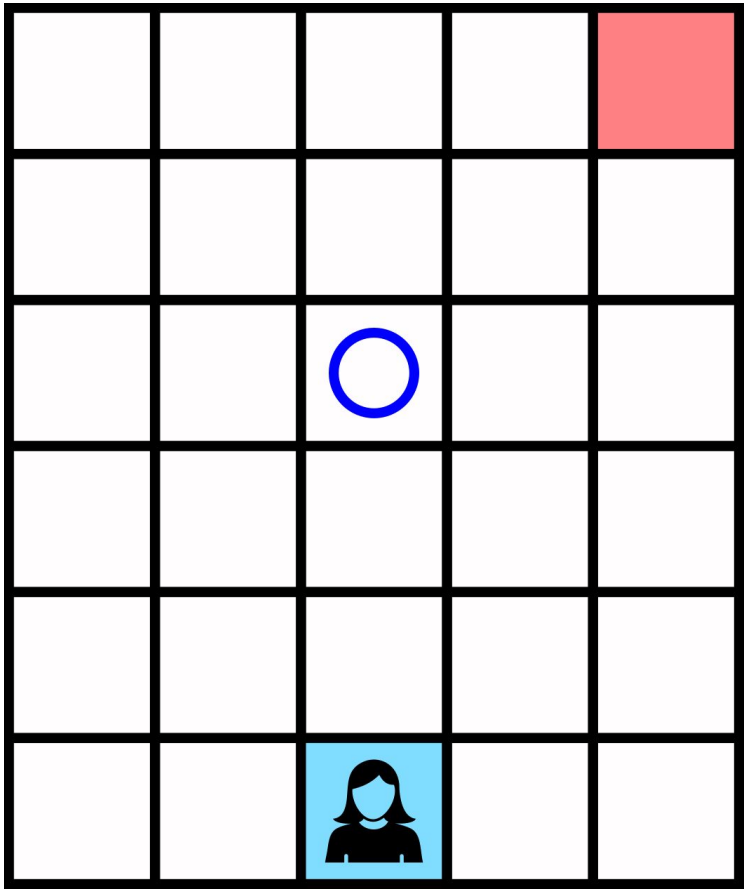
MC evaluation



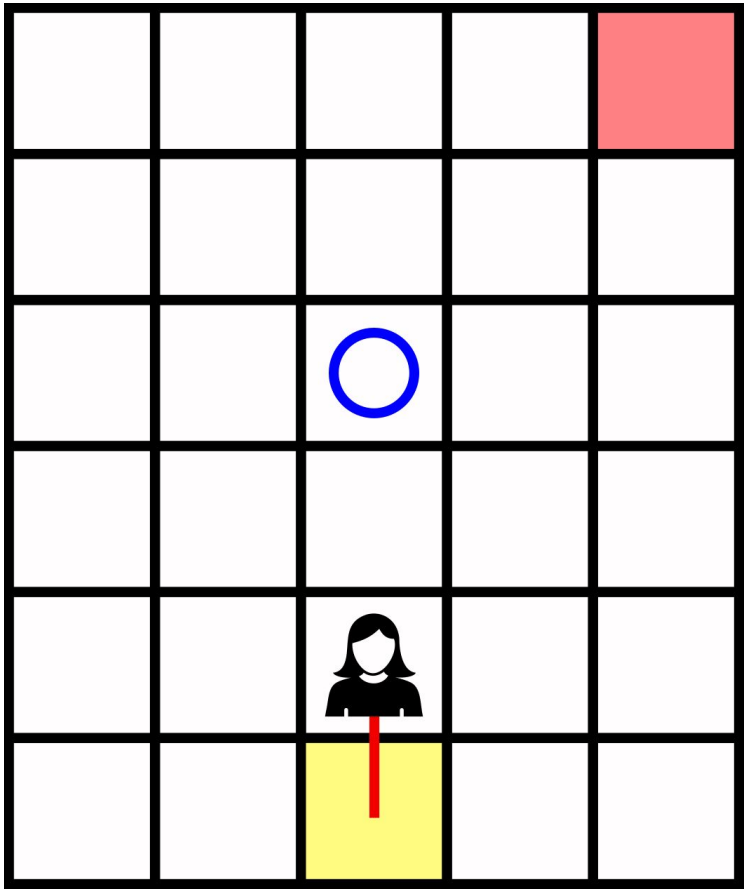
MC evaluation



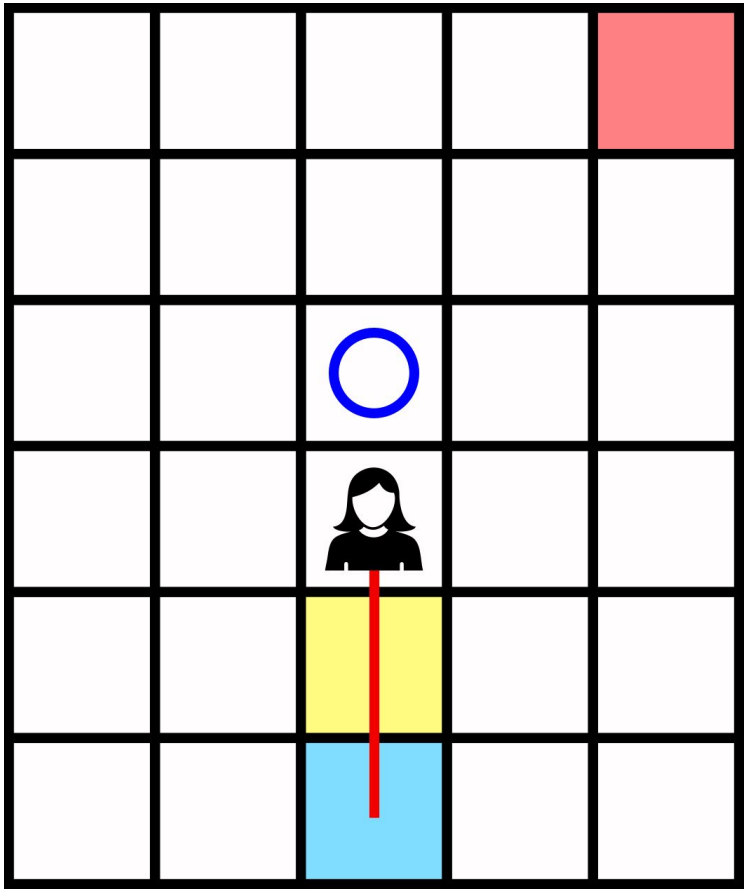
TD evaluation



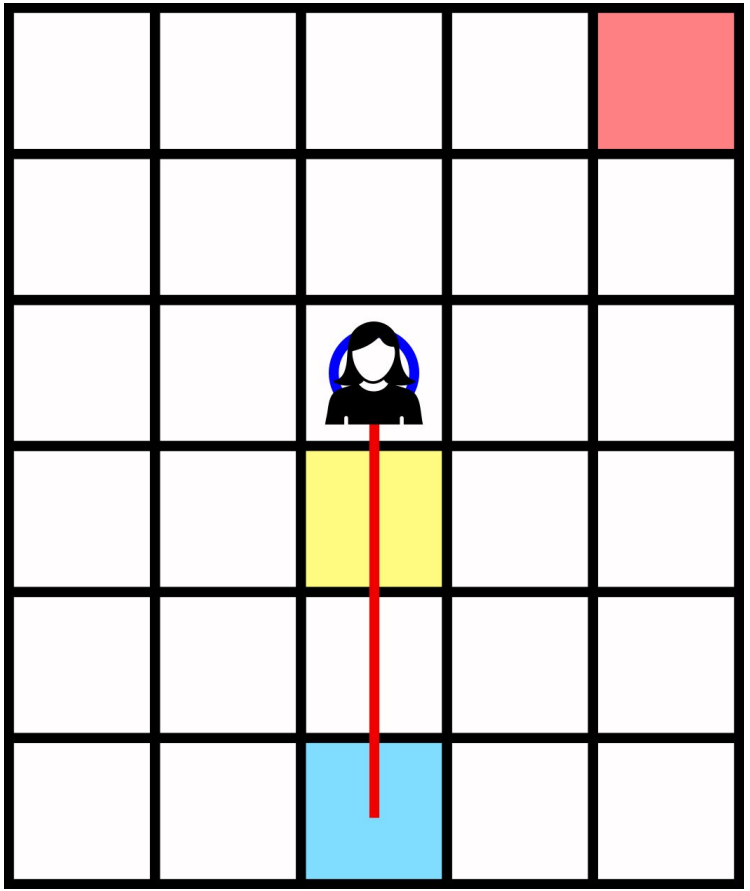
TD evaluation



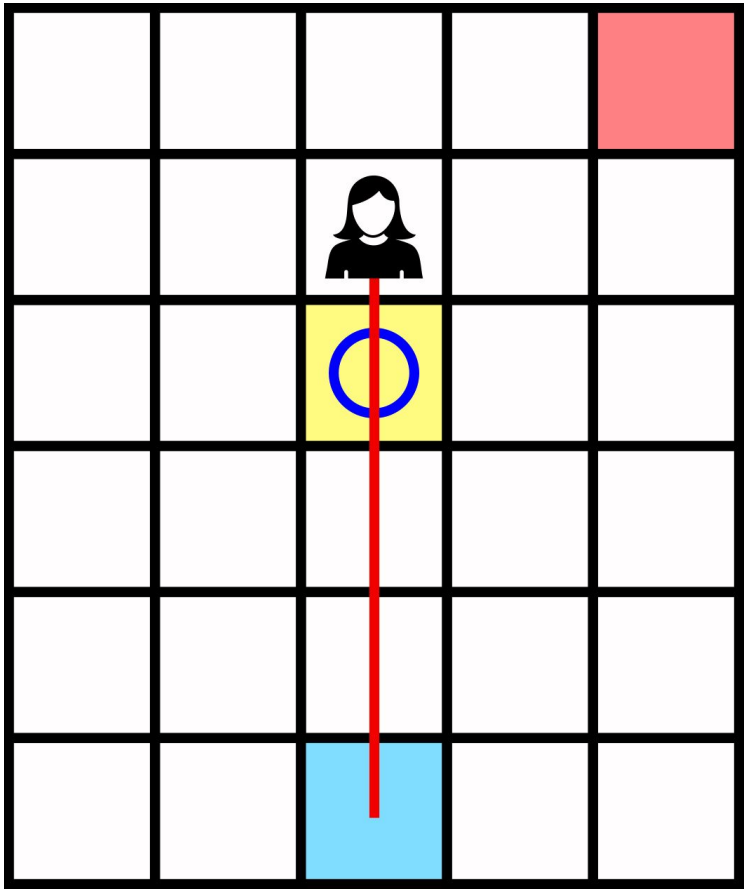
TD evaluation



TD evaluation



TD evaluation



TD prediction

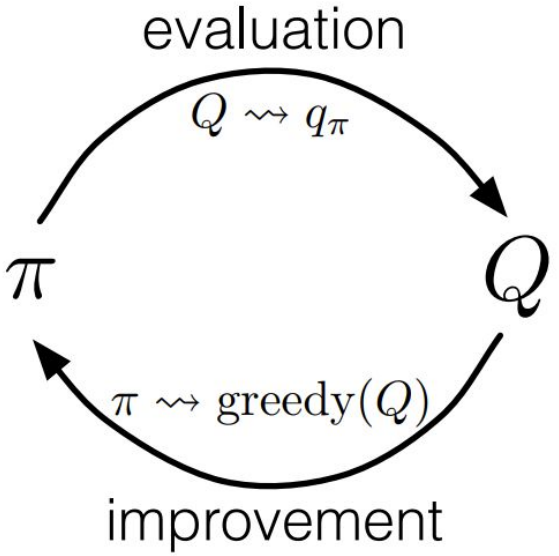
Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\mathit{terminal}) = 0$

Loop for each episode:
    Initialize S
    Loop for each step of episode:
        $A \leftarrow$ action given by π for S
        Take action A, observe R, S'
        $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
        $S \leftarrow S'$
    until S is terminal

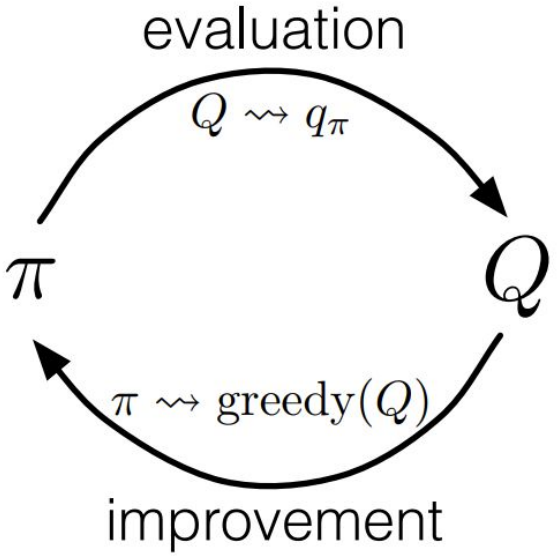
Finding good policies with TD

- Can use generalized policy iteration again!



Finding good policies with TD

- Can use generalized policy iteration again!
- Change **evaluation** to use TD error
- Example, TD(0) for actions:



$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

- **Sarsa**

Sarsa

Sarsa for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
    Initialize S
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Loop for each step of episode:
        Take action A, observe R, S'
        Choose A' from S' using policy derived from Q (e.g., ε-greedy)
        $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
        $S \leftarrow S'$; $A \leftarrow A'$;
    until S is terminal

Off-policy TD control

- One-step off-policy TD method using greedy target policy?

Off-policy TD control

- One-step off-policy TD method using greedy target policy?

Q-learning

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
    Initialize S
    Loop for each step of episode:
        Choose A from S using policy derived from Q (e.g., ε-greedy)
        Take action A, observe R, S'
        $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
        $S \leftarrow S'$
    until S is terminal

Expected SARSA

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \, Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \, \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Other possibilities?

Expected SARSA

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma\, Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a|s_{t+1}) Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Expected SARSA

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \Sigma_a \pi(a|s_{t+1}) Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Uses **expected value** of estimated value of s_{t+1}

Can be used off-policy as well!

What if target policy is greedy?

Expected SARSA

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \Sigma_a \pi(a|s_{t+1}) Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

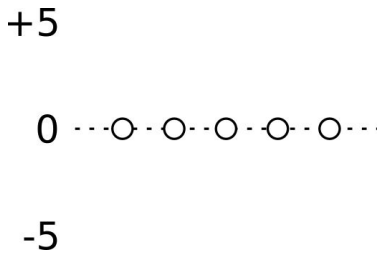
Uses **expected value** of estimated value of s_{t+1}

Can be used off-policy as well!

What if target policy is greedy? Q-learning!

Maximization bias

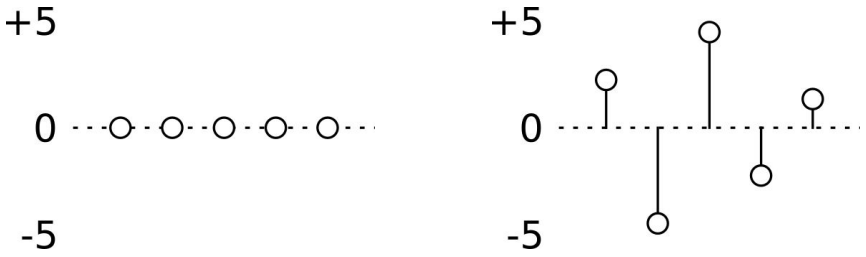
- **Maximization** is often used
  - Q-learning, ε-greedy
- When estimating values -> **maximization bias**



True value: 0

Maximization bias

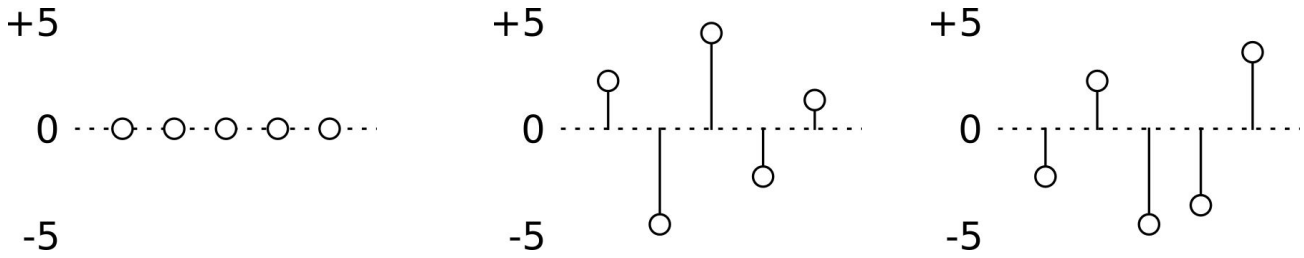
- **Maximization** is often used
  - Q-learning, ε-greedy
- When estimating values -> **maximization bias**



True value: 0 Est. value: 4.5

Maximization bias

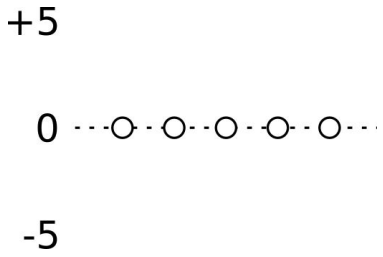
- **Maximization** is often used
  - Q-learning, ε-greedy
- When estimating values -> **maximization bias**



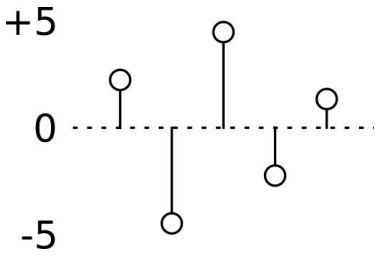
True value: 0 Est. value: 4.5 Est. value: 4

Maximization bias

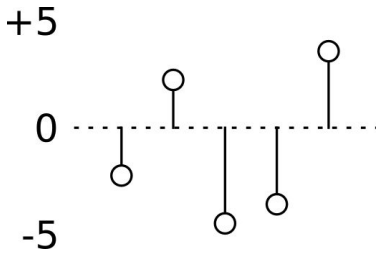
- **Maximization** is often used
  - Q-learning, ε-greedy
- When estimating values -> **maximization bias**



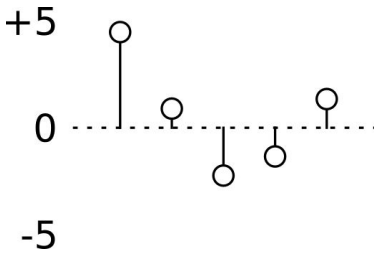
True value: 0



Est. value: 4.5

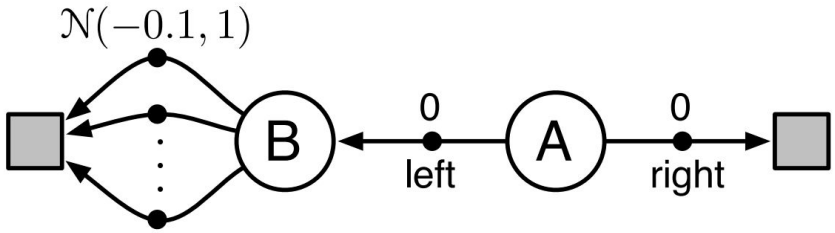


Est. value: 4

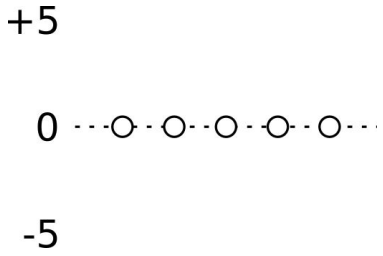


Est. value: 4.25

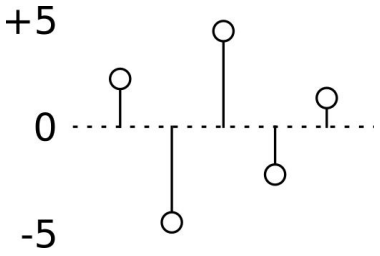
Maximization bias



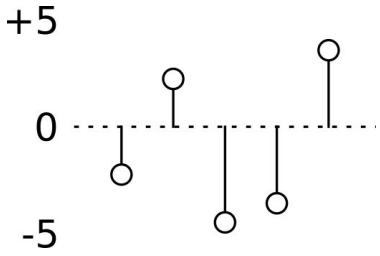
- **Maximization** is often used
  - Q-learning, ε-greedy
- When estimating values -> **maximization bias**



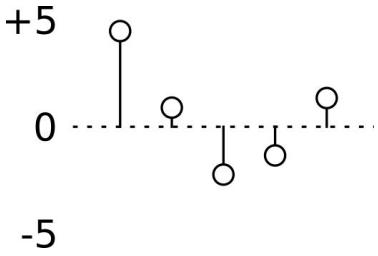
True value: 0



Est. value: 4.5



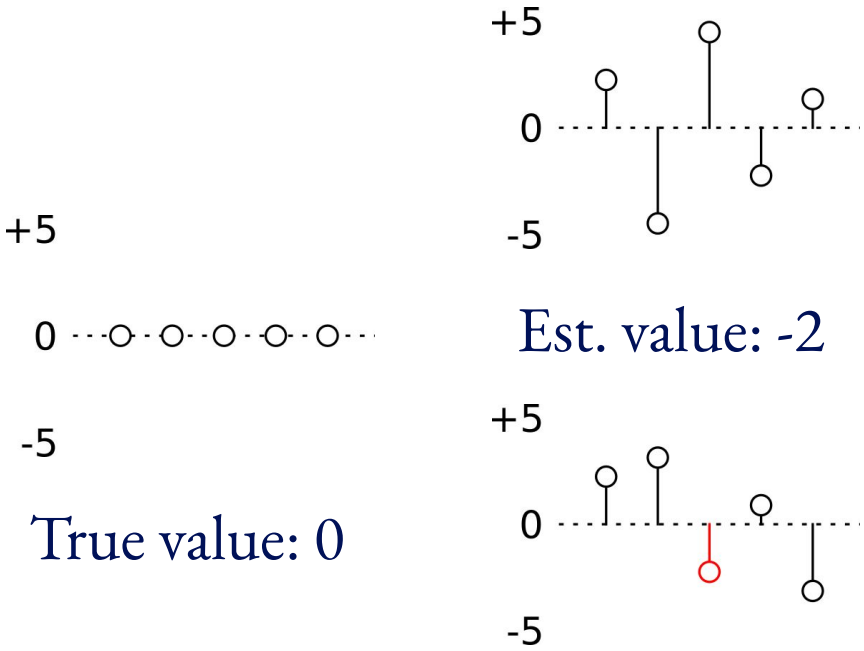
Est. value: 4



Est. value: 4.25

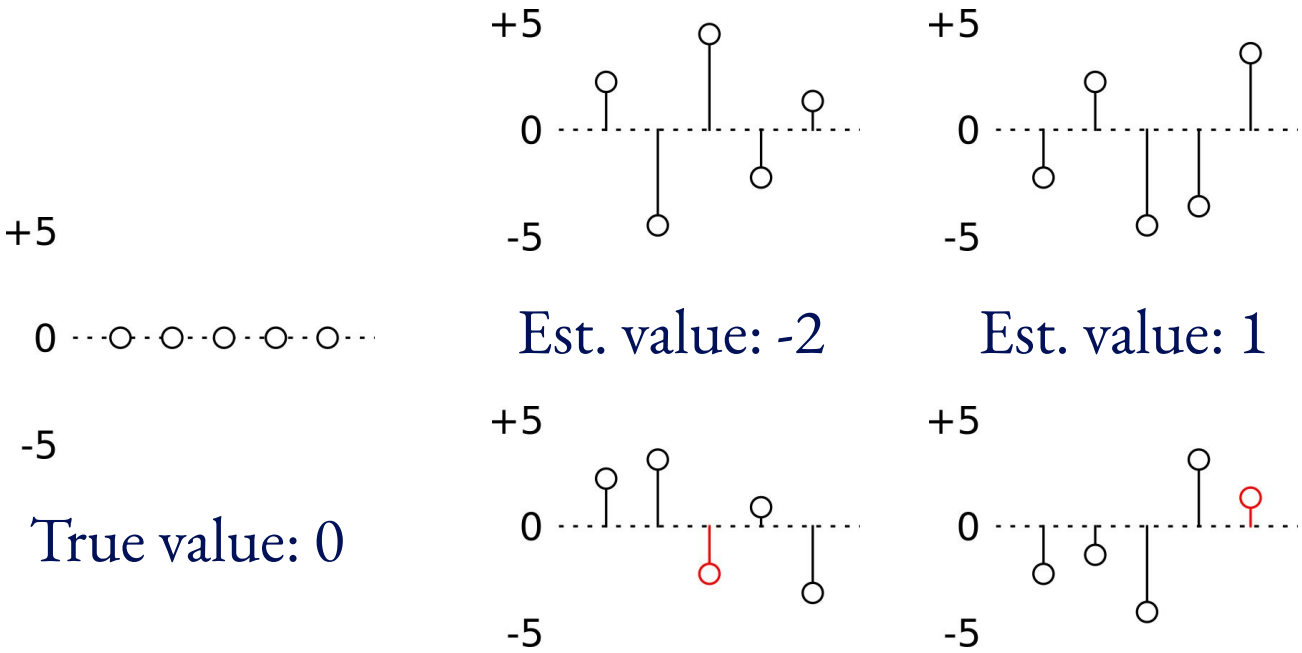
Maximization bias

- Solution: **Double learning** -> use 2 independent Q tables



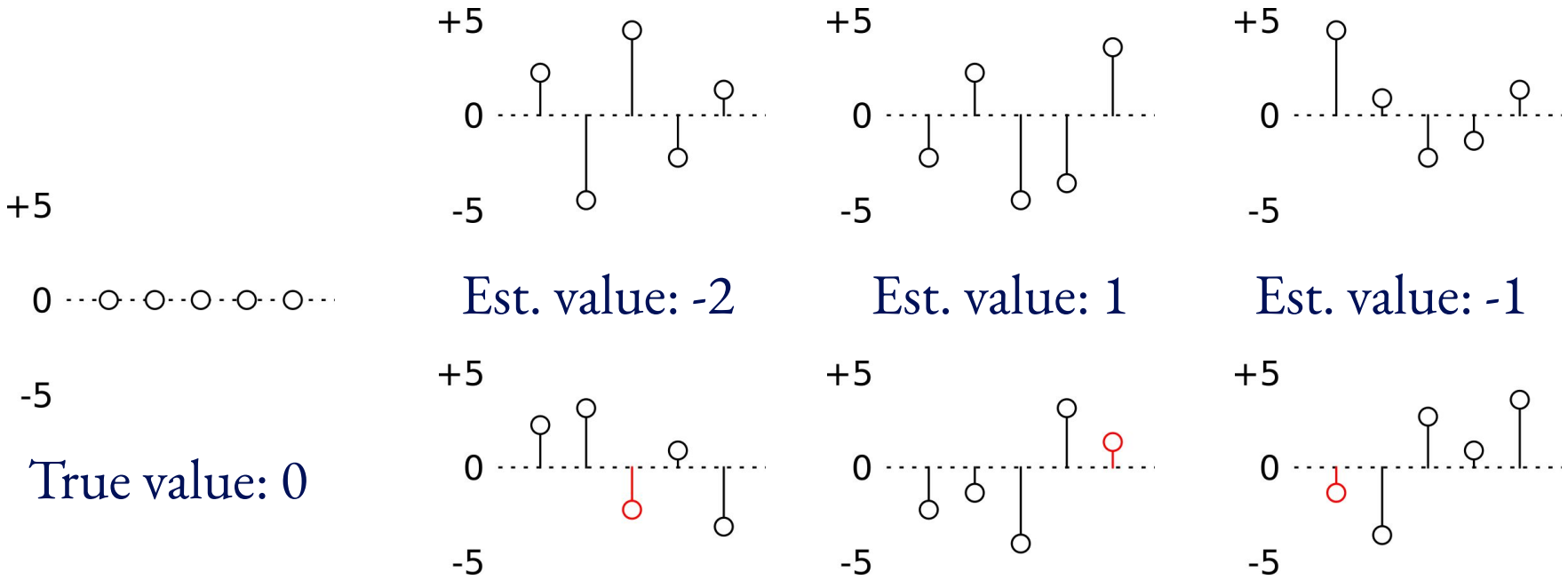
Maximization bias

- Solution: **Double learning** -> use 2 independent Q tables

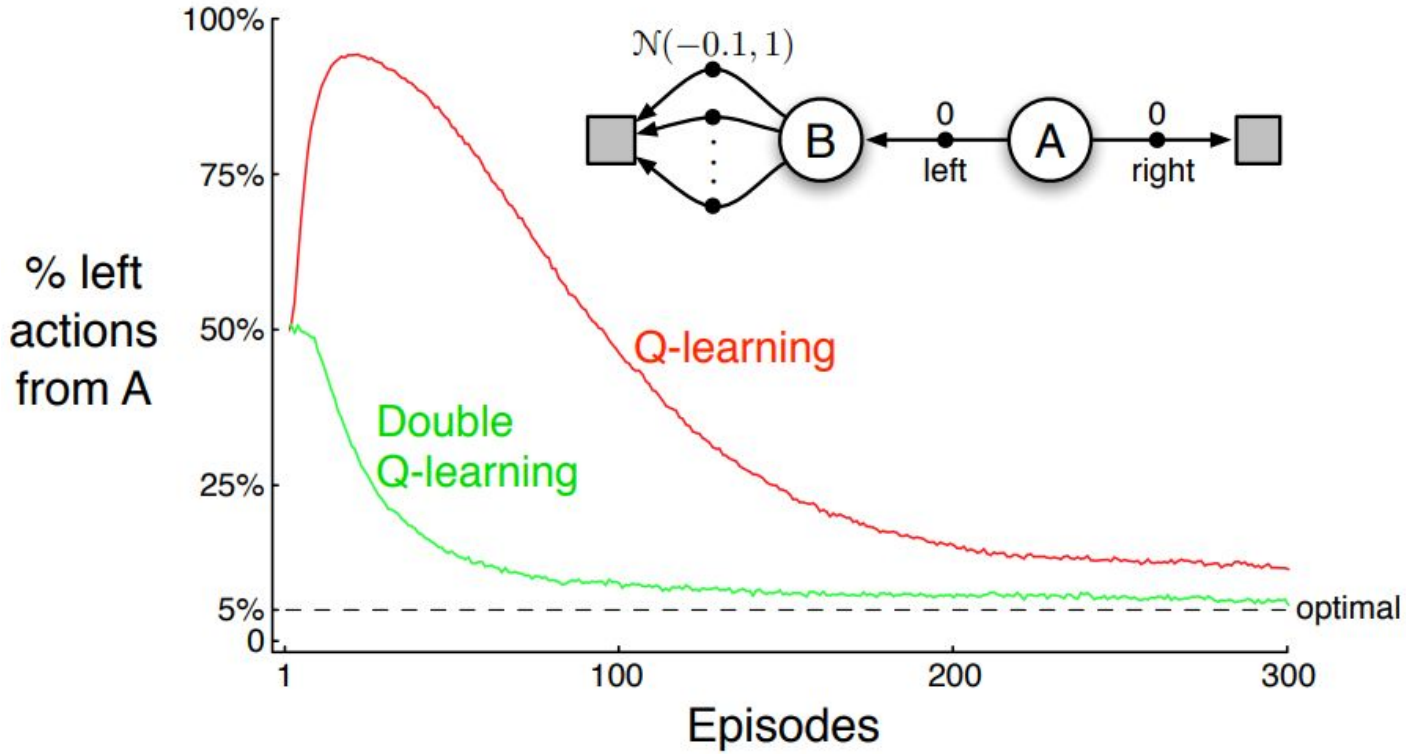


Maximization bias

- Solution: **Double learning** -> use 2 independent Q tables



Maximization bias



Maximization bias

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

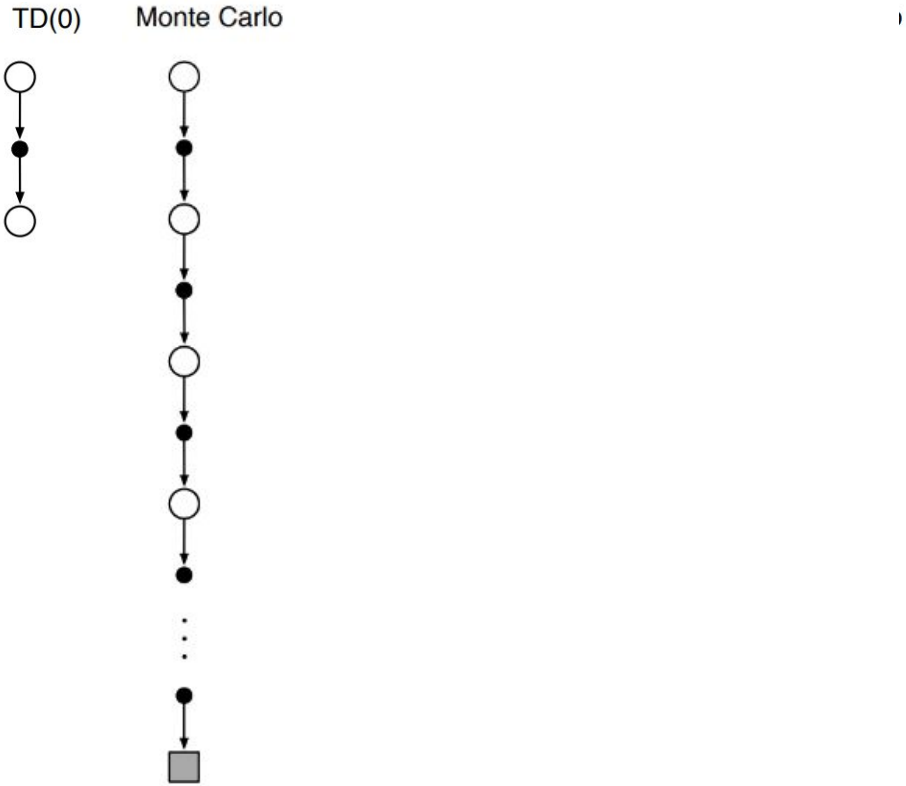
$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \operatorname{argmax}_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

    else:

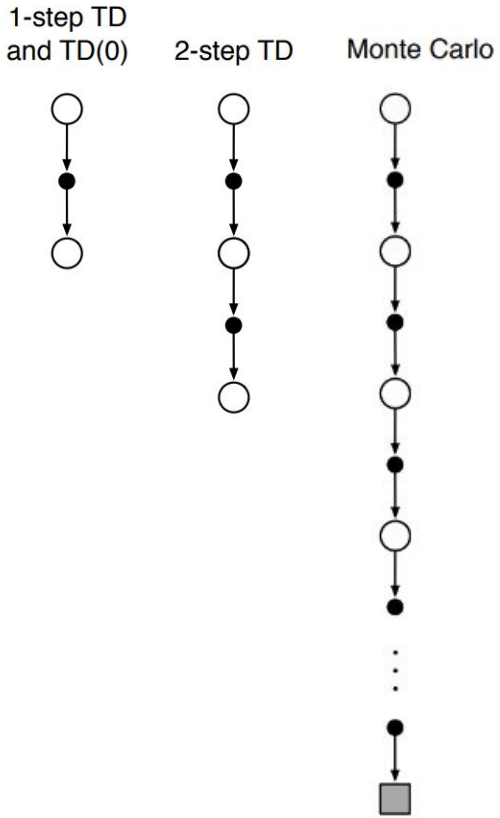
$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \operatorname{argmax}_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal

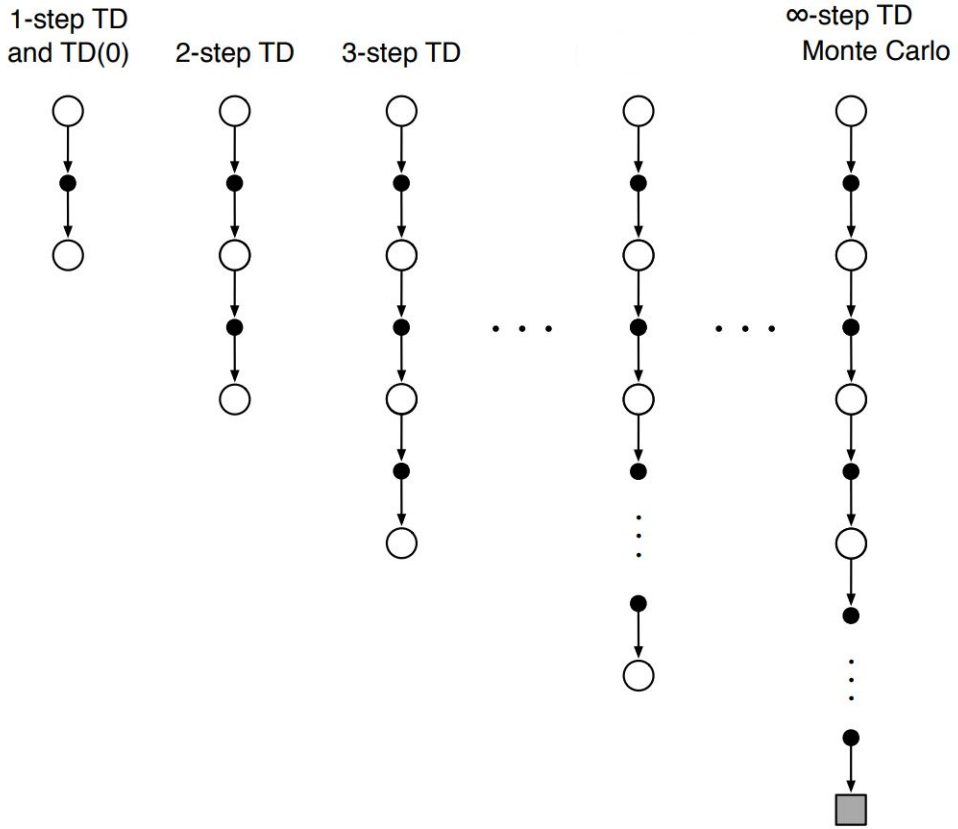
TD vs MC



N-step TD prediction



N-step TD prediction



N-step TD prediction

MC return:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$$

1-step TD return:

$$G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1})$$

2-step TD return:

?

N-step TD prediction

MC return:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$$

1-step TD return:

$$G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1})$$

2-step TD return:

$$G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$$

N-step TD prediction

2-step TD return:

$$G_{t:t+2} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$$

n-step TD return:

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$$

Estimate **value of states**:

$$V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha \big[G_{t:t+n} - V_{t+n-1}(S_t) \big]$$

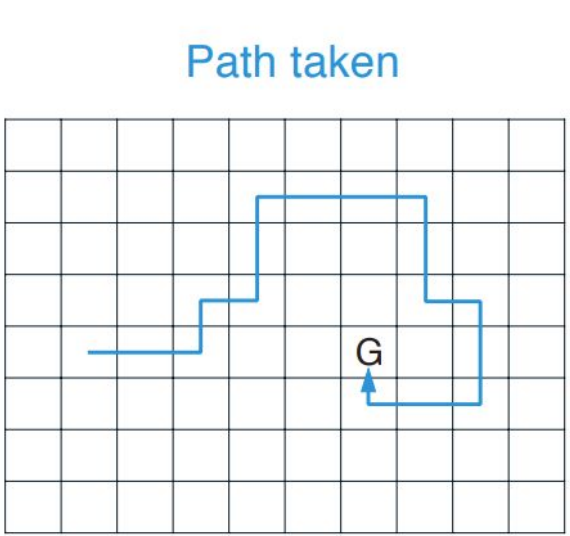
N-step SARSA

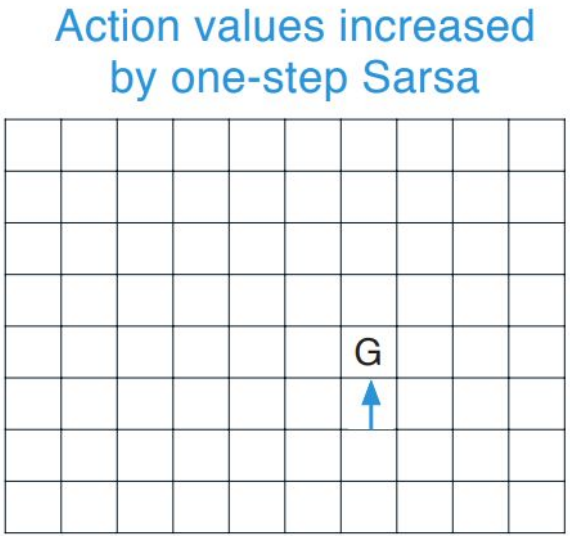
Similar to 1-step SARSA, just use **n-step return**

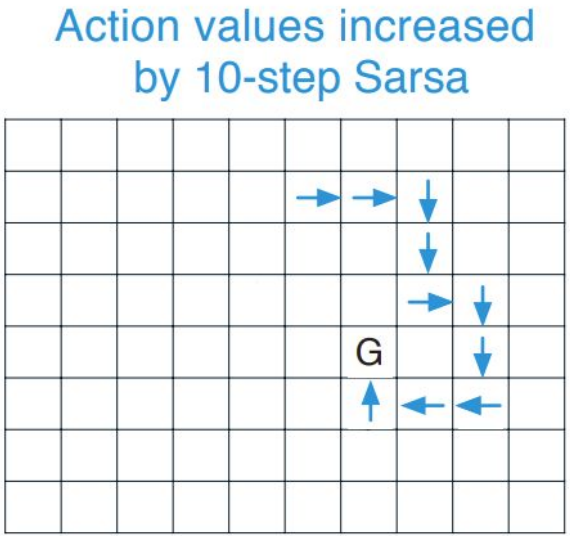
$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^{n} Q_{t+n-1}(S_{t+n}, A_{t+n})$$

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha \left[G_{t:t+n} - Q_{t+n-1}(S_t, A_t) \right]$$

N-step SARSA

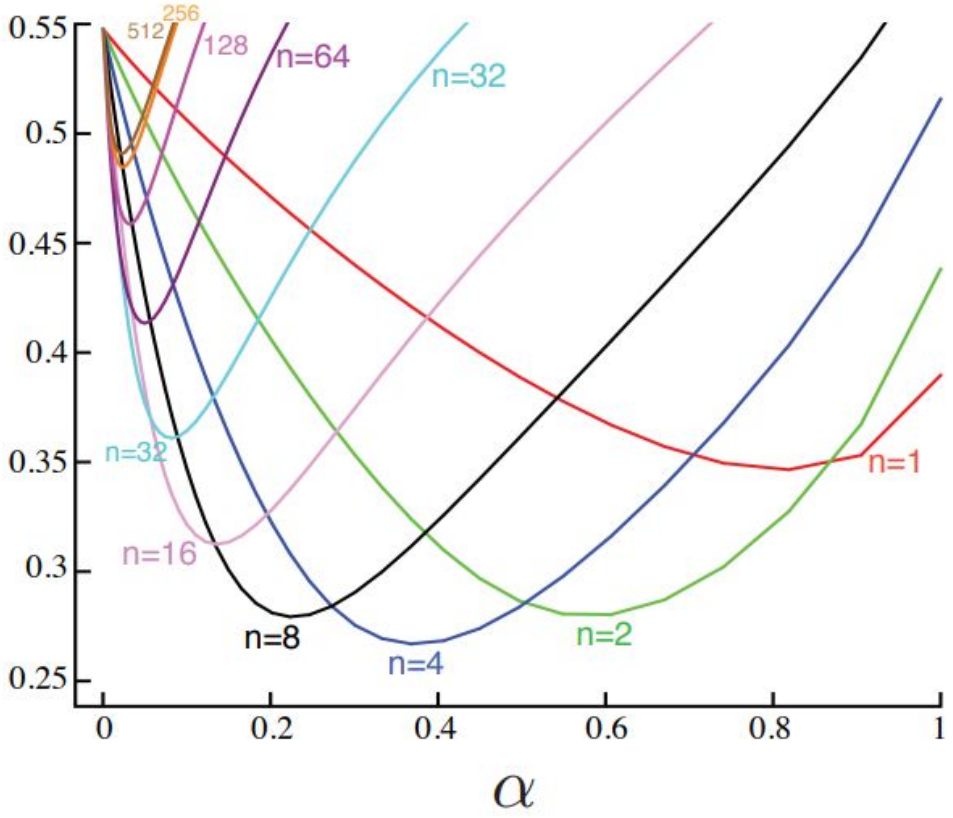

Path taken
G


Action values increased by one-step Sarsa
G


Action values increased by 10-step Sarsa
G

N-step SARSA

Average RMS error over 19 states and first 10 episodes



Summary

- We can learn from experience!
- Basic approach: generalized policy iteration
- Ingredients: policy evaluation and policy improvement
- Evaluation after episode using return: Monte-Carlo
- Evaluation after n steps using estimated target: Temporal-Difference
- One policy for generating episodes and updating: on-policy
- Different behaviour policy and target policy: off-policy
- Examples: Sarsa (on-policy TD), Q-Learning (off-policy TD)

Next weeks

- Read Sutton & Barto, chapter 5 and 6, and 7.1 - 7.2
- Work on assignment 2: SARSA, Q-Learning, Expected SARSA
  - Any questions?